

CATEDRA DE ARQUITECTURA DE COMPUTADORAS

REPRESENTACION DE DATOS

Introducción

- En **INFORMATICA (Información Automática)**, se trabaja con un conjunto de datos para elaborarlo, procesarlo y obtener un resultado en un proceso de:

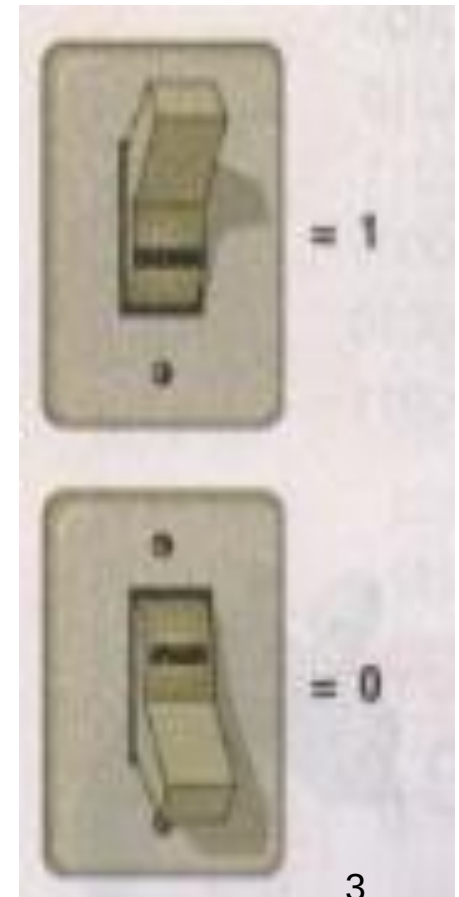
ENTRADA -> MEMORIA -> PROCESO -> SALIDA

Elaboración: Se produce en el interior de la máquina, y esta funciona con energía eléctrica.

Es por ello que necesitamos analizar como se adapta la matemática y la aritmética para que pueda funcionar con energía eléctrica.

Sistemas de Representación

- Las computadoras representan todos sus datos en sistema de numeración binario.
- Los datos viajan, se procesan y se almacenan en las computadoras a través de impulsos eléctricos. Estos impulsos se representan por dos estados:
 - ✓ Encendido o apagado = 1 ó 0

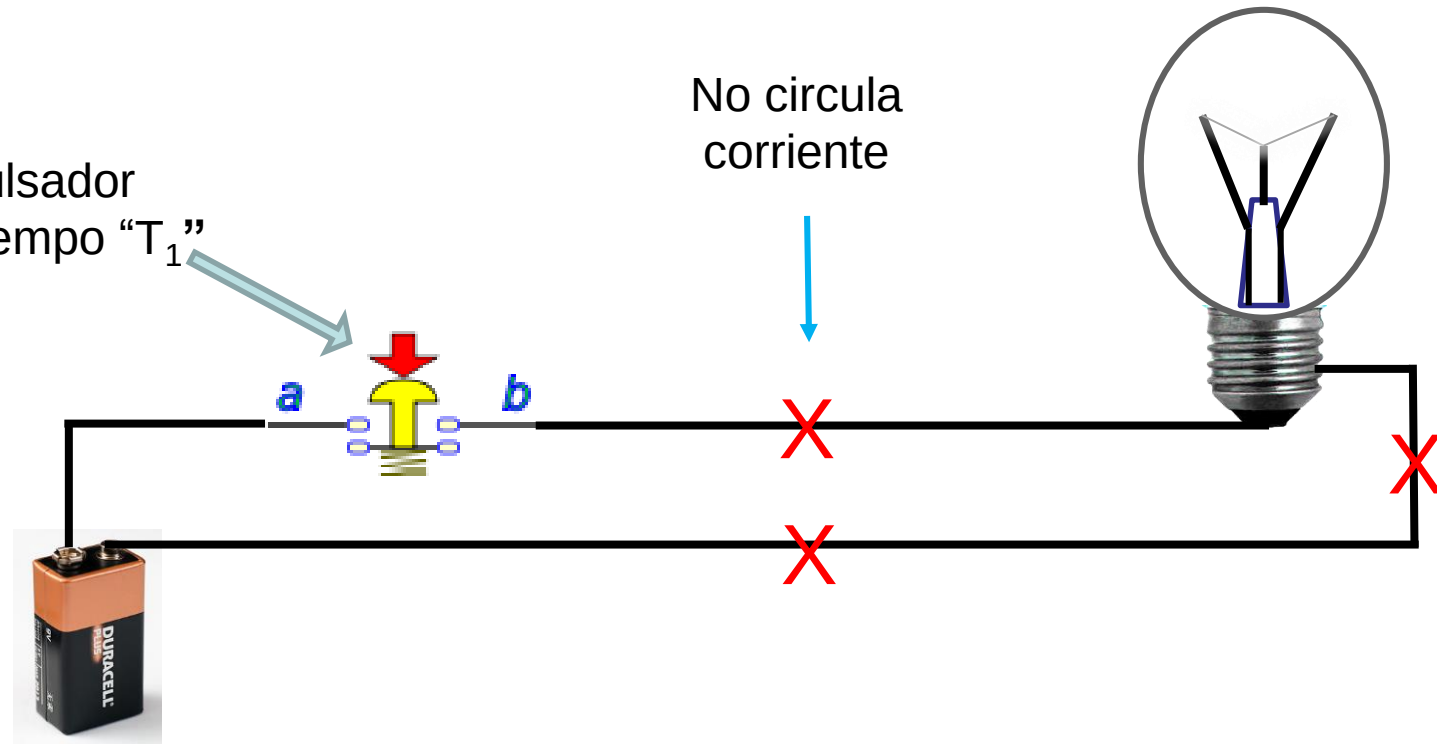


Sistemas de Representación

❑ EJEMPLO 1:

- ✓ SI NO PRESIONAMOS EL PULSADOR DURANTE UN TIEMPO T_1 . NO CIRCULARÁ CORRIENTE Y LA LÁMPARA PERMANECERÁ APAGADA DURANTE ESE TIEMPO T_1 .

NO presionar el pulsador
Durante el intervalo tiempo " T_1 "

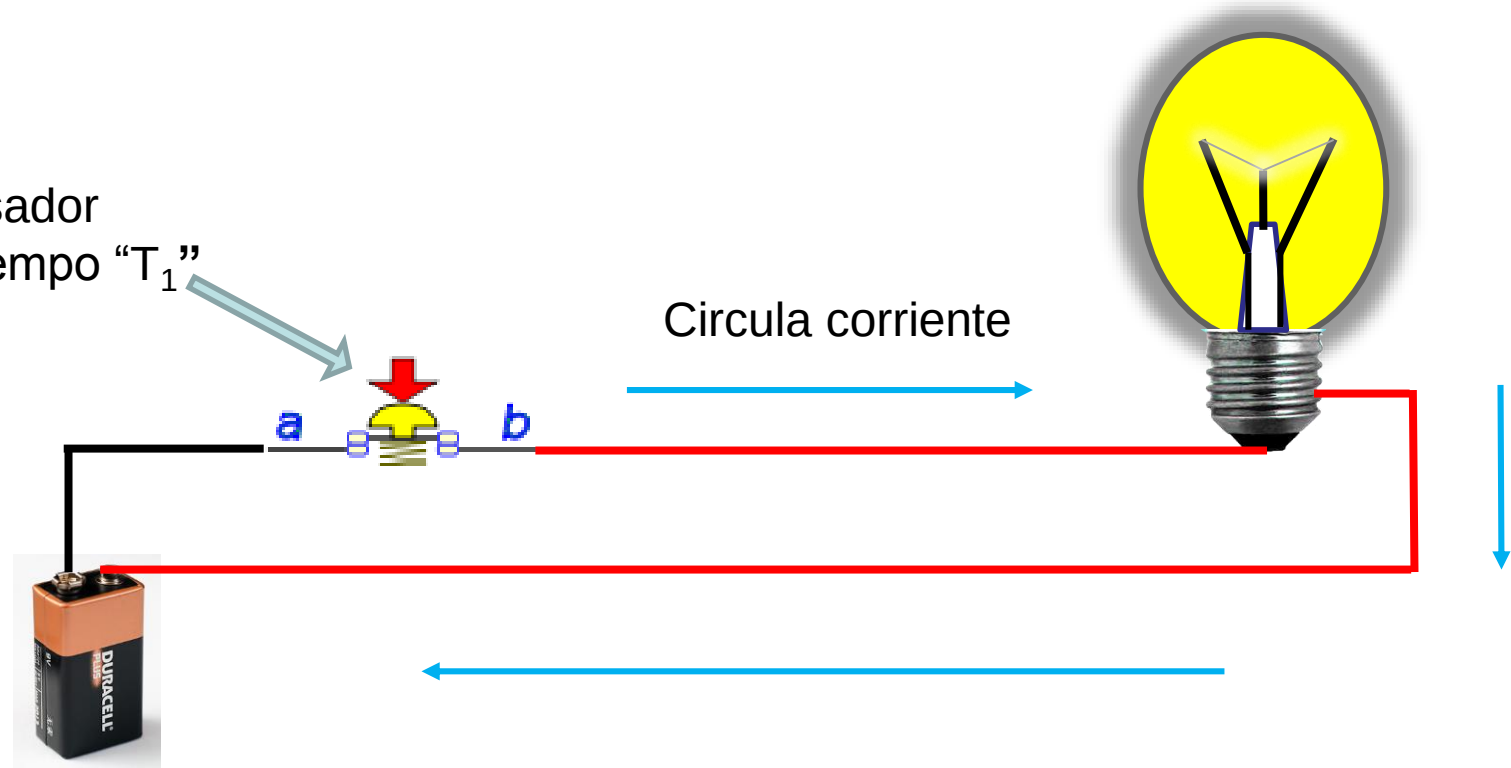


Sistemas de Representación

❑ EJEMPLO 2:

- ✓ SI PRESIONAMOS EL PULSADOR DURANTE UN TIEMPO T_1 . CIRCULARÁ CORRIENTE Y LA LÁMPARA PERMANECERÁ ENCENDIDA DURANTE ESE TIEMPO T_1 .

SI Presionar pulsador durante el intervalo tiempo " T_1 "



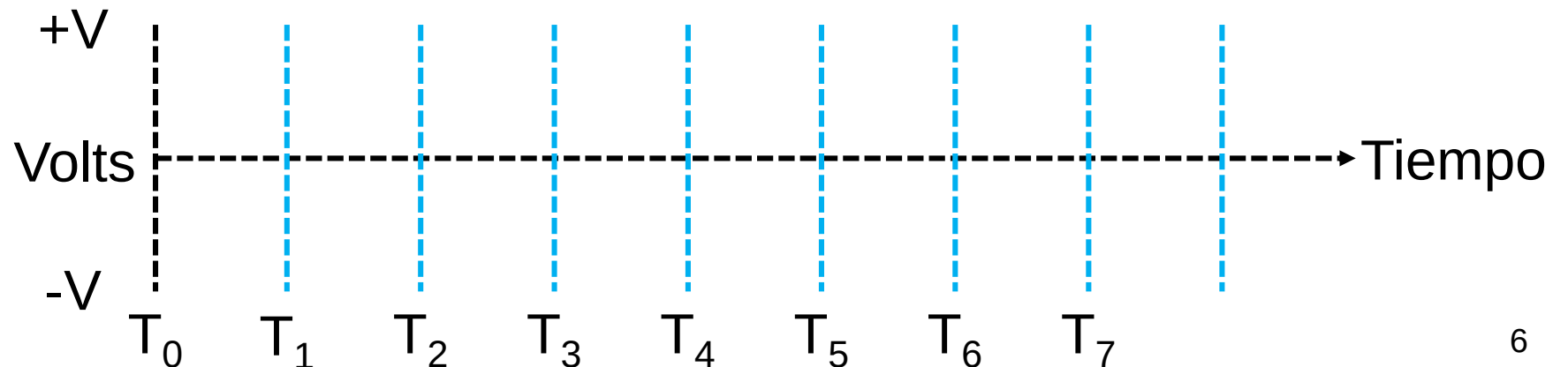
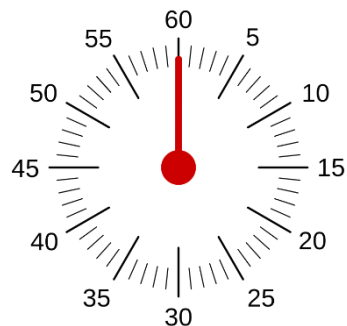
Sistemas de Representación

❑ EJEMPLO 3: SINCRONIZAR LA OPERATORIA:

- ✓ LA ACCIÓN QUE DECIDAMOS REALIZAR, YA SEA PRESIONAR EL PULSADOR O DEJARLO SIN PRESIONAR, DEBERÁ MANTENERSE CONSTANTE DURANTE UN TIEMPO T_i . AL FINALIZAR, ARBITRAMOS LA SIGUIENTE ACCIÓN Y DEBEREMOS MANTENERLA CONSTANTE DURANTE EL TIEMPO T_{i+1} Y ASÍ SUCESIVAMENTE.
- ✓ LOS INTERVALOS DE TIEMPO SON FIJOS, O SEA $T_1 = T_2 = \dots T_N$.

SUPONGAMOS:

$$0 - 60 = T_i$$

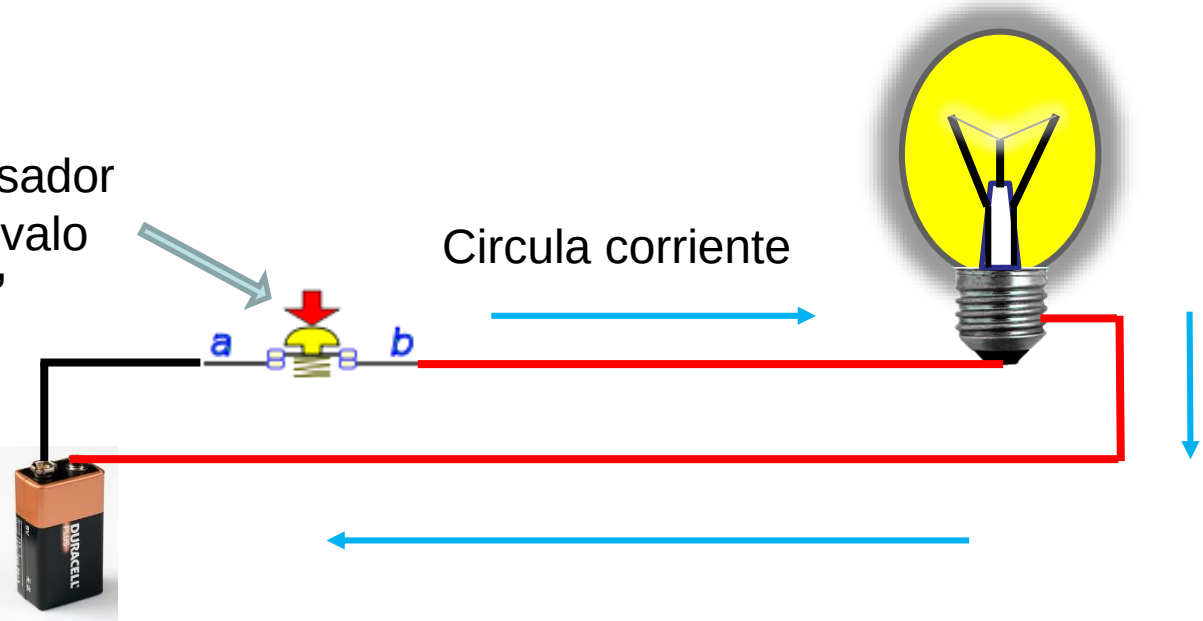


Sistemas de Representación

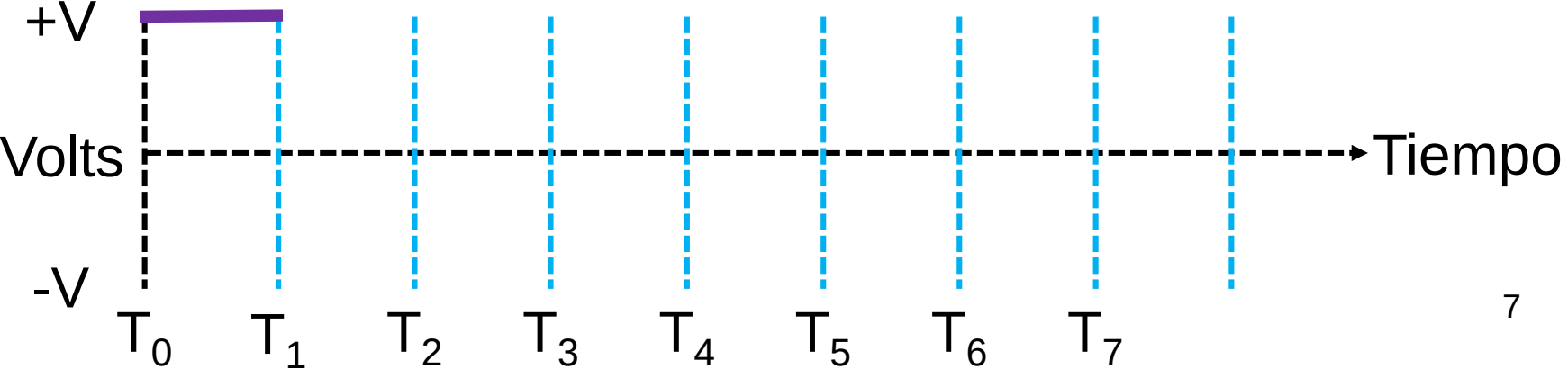
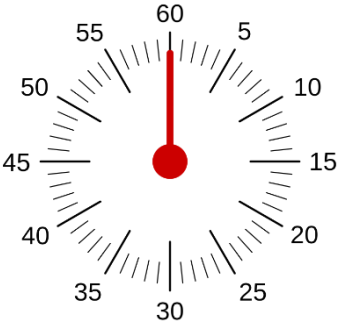
☐ ✓ ACCIÓN 1:

PULSAR SI / NO	PASO DE CORRIENTE
SI	✓

SI Presionar pulsador
durante el intervalo
tiempo " T_1 "



SUPONGAMOS:
 $0 - 60 = T_i$

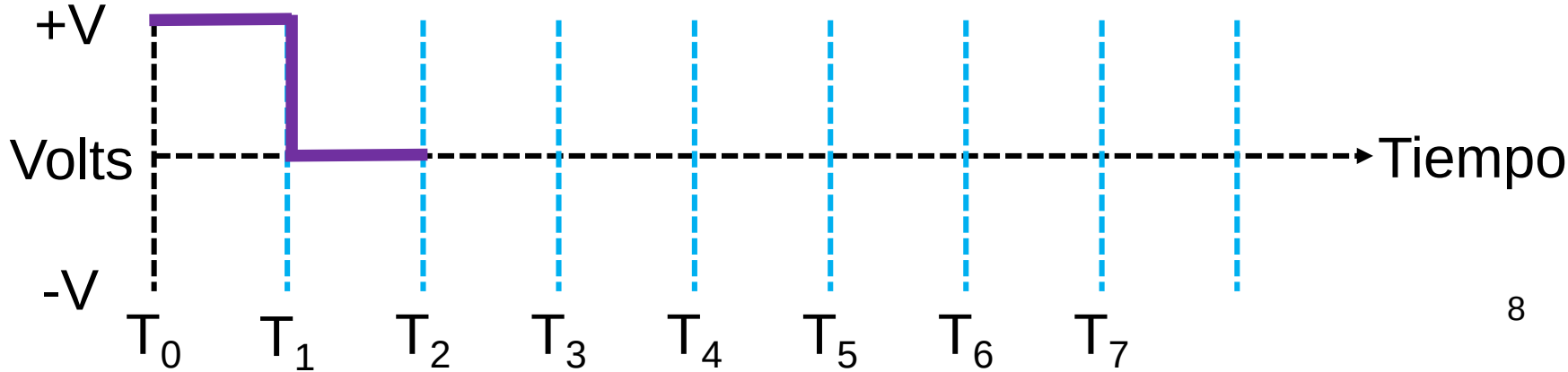
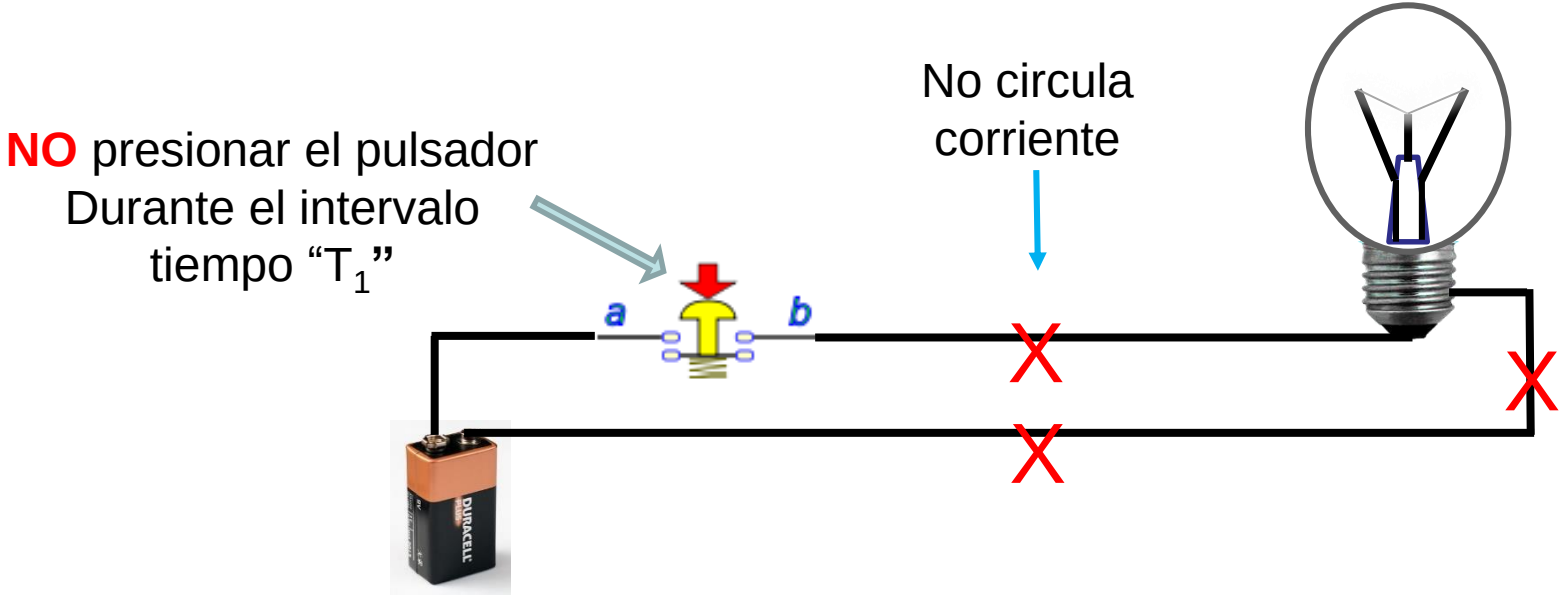
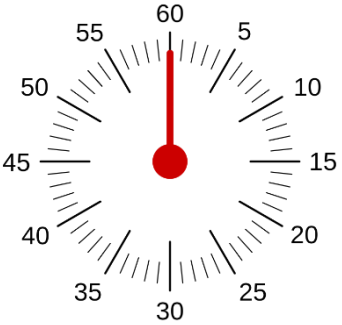


Sistemas de Representación

☐ ✓ ACCIÓN 2:

PULSAR SI / NO	PASO DE CORRIENTE
SI	✓
NO	X

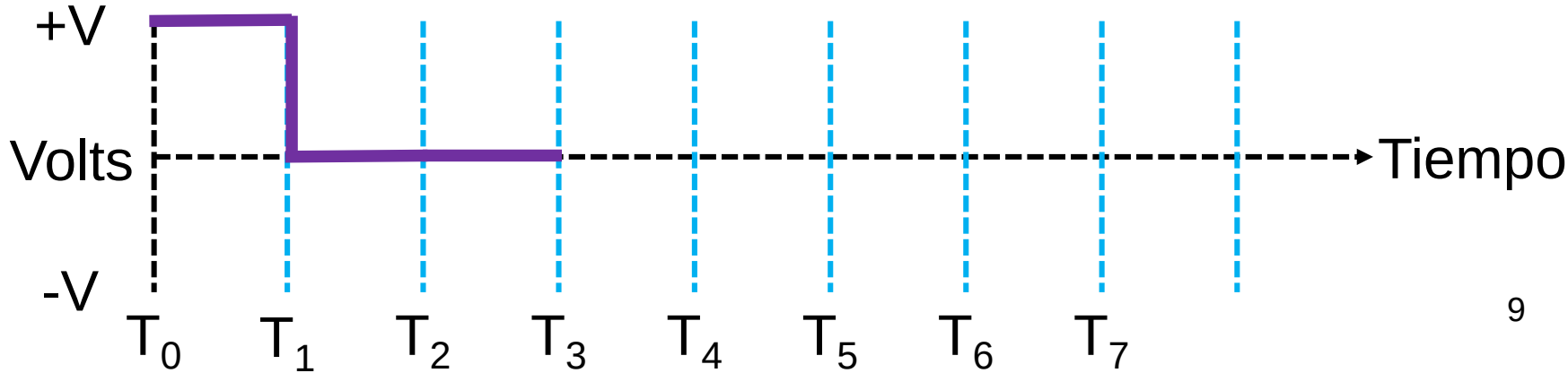
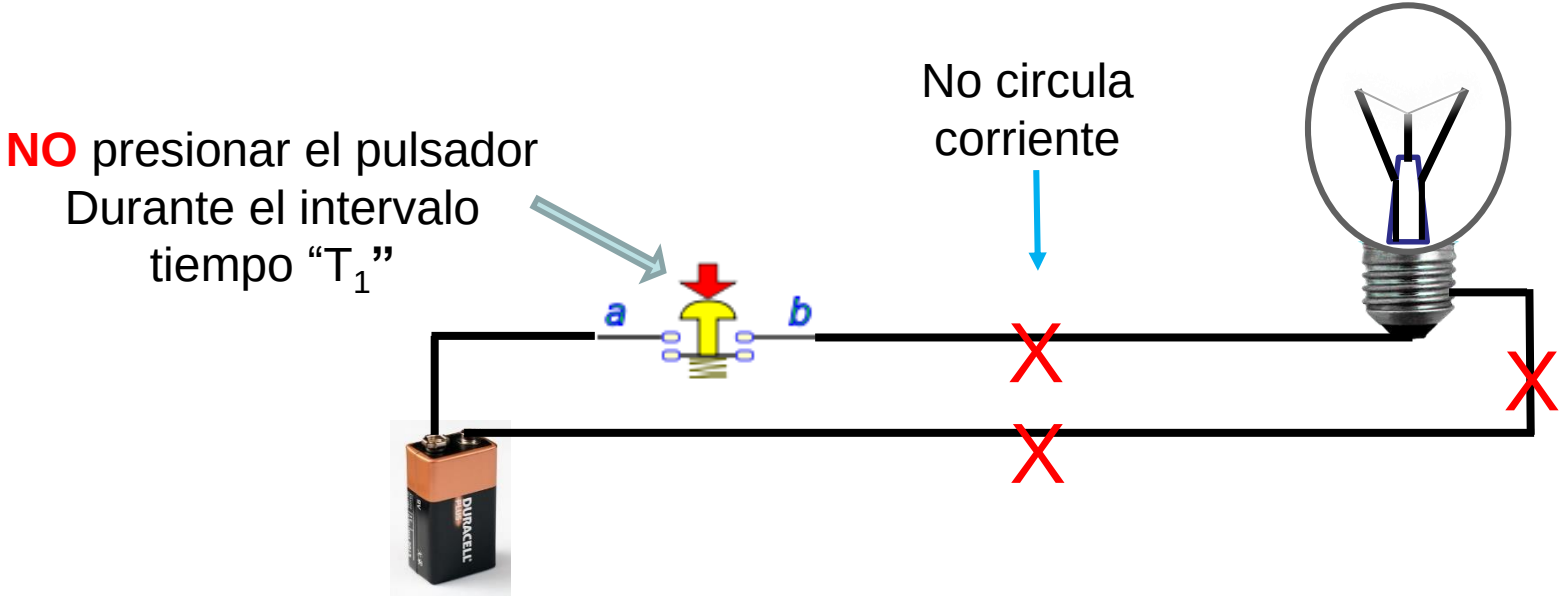
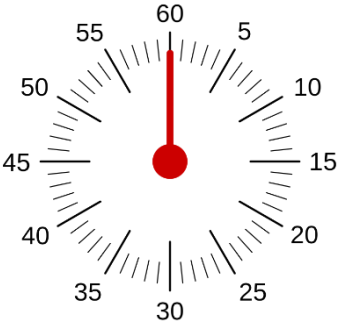
SUPONGAMOS:
 $0 - 60 = T_i$



Sistemas de Representación

☐ ✓ ACCIÓN 3:

PULSAR SI / NO	PASO DE CORRIENTE
SI	✓
NO	X
NO	X

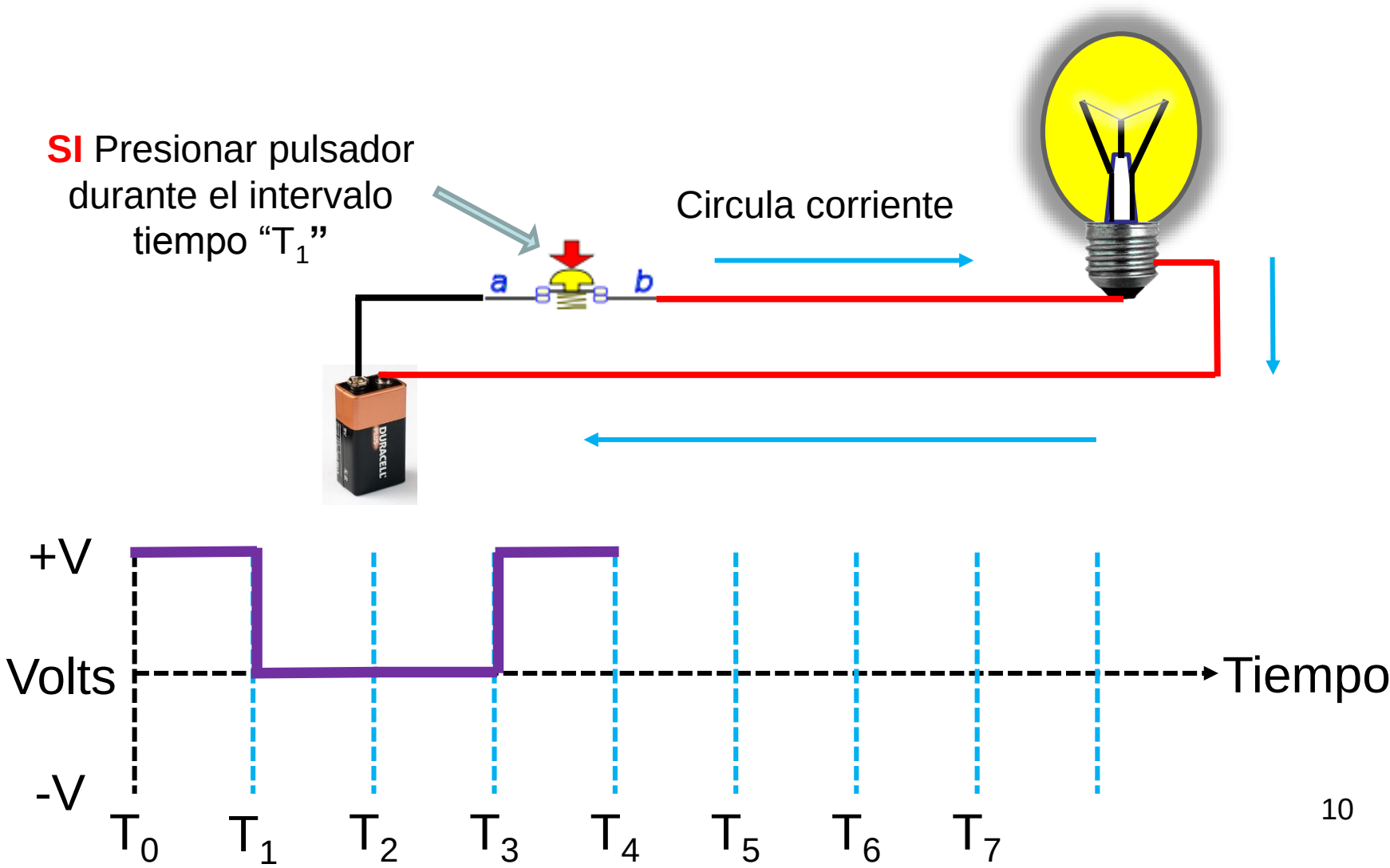
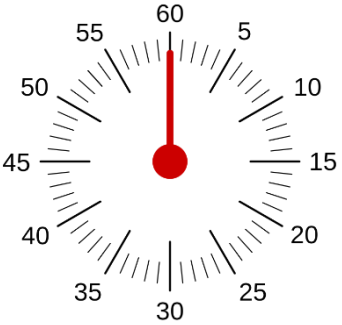


Sistemas de Representación

☐ ✓ ACCIÓN 4:

PULSAR SI / NO	PASO DE CORRIENTE
SI	✓
NO	X
NO	X
SI	✓

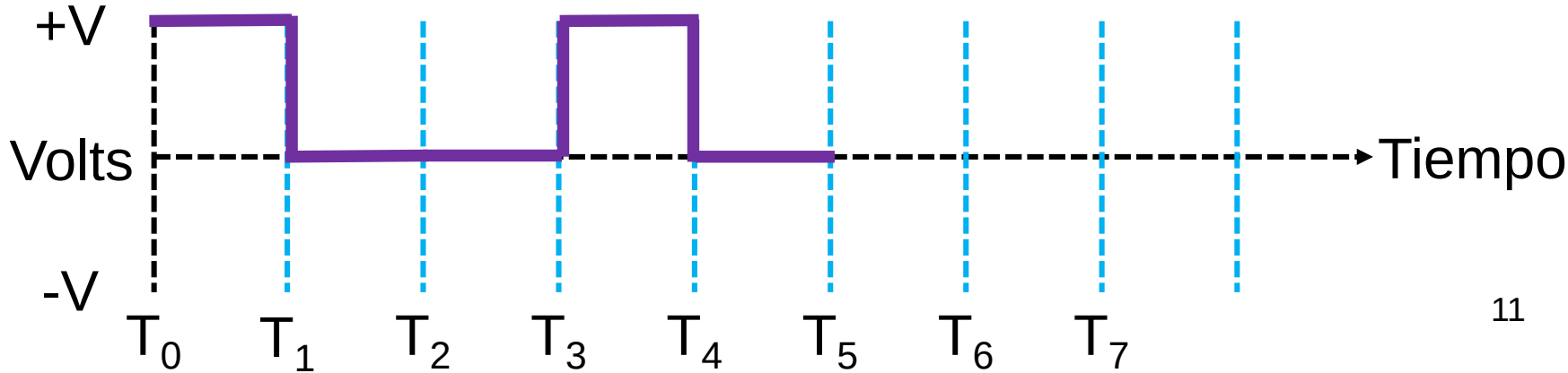
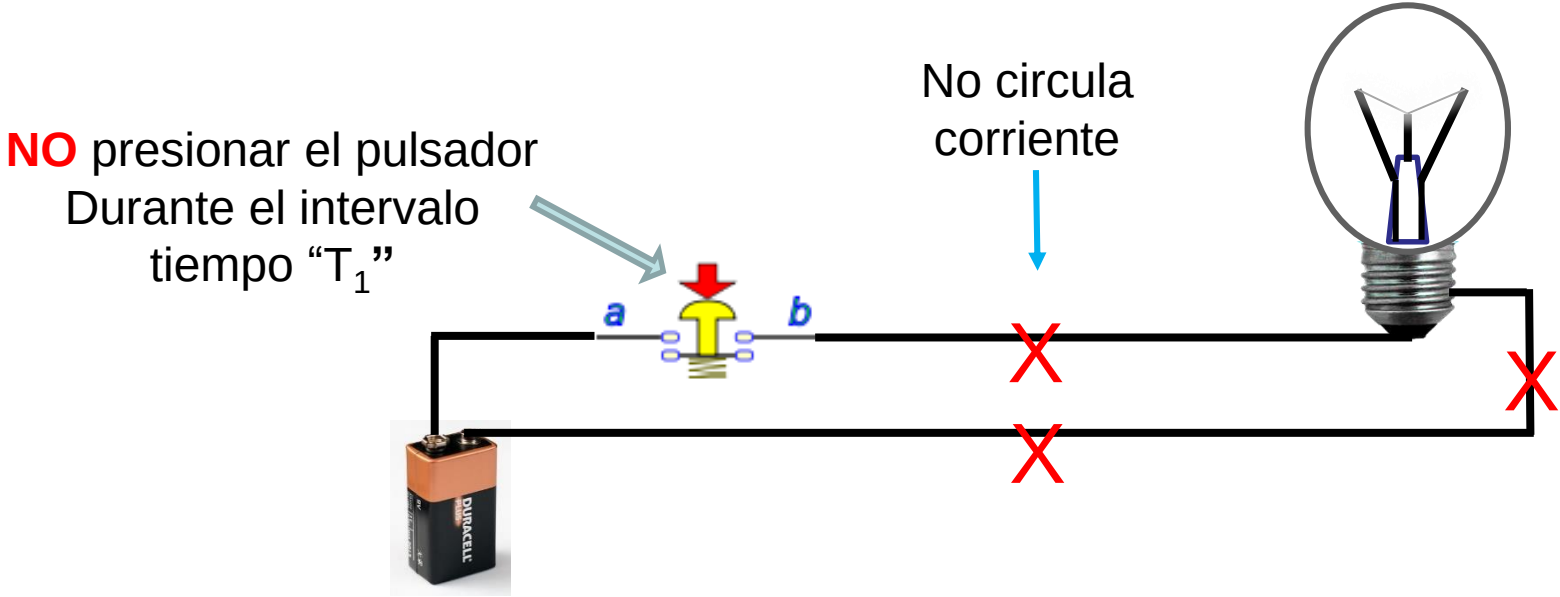
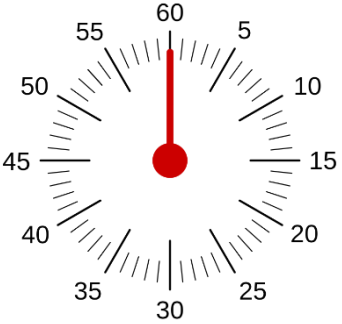
SUPONGAMOS:
 $0 - 60 = T_i$



Sistemas de Representación

☐ ✓ ACCIÓN 5:

PULSAR SI / NO	PASO DE CORRIENTE
SI	✓
NO	X
NO	X
SI	✓
NO	X

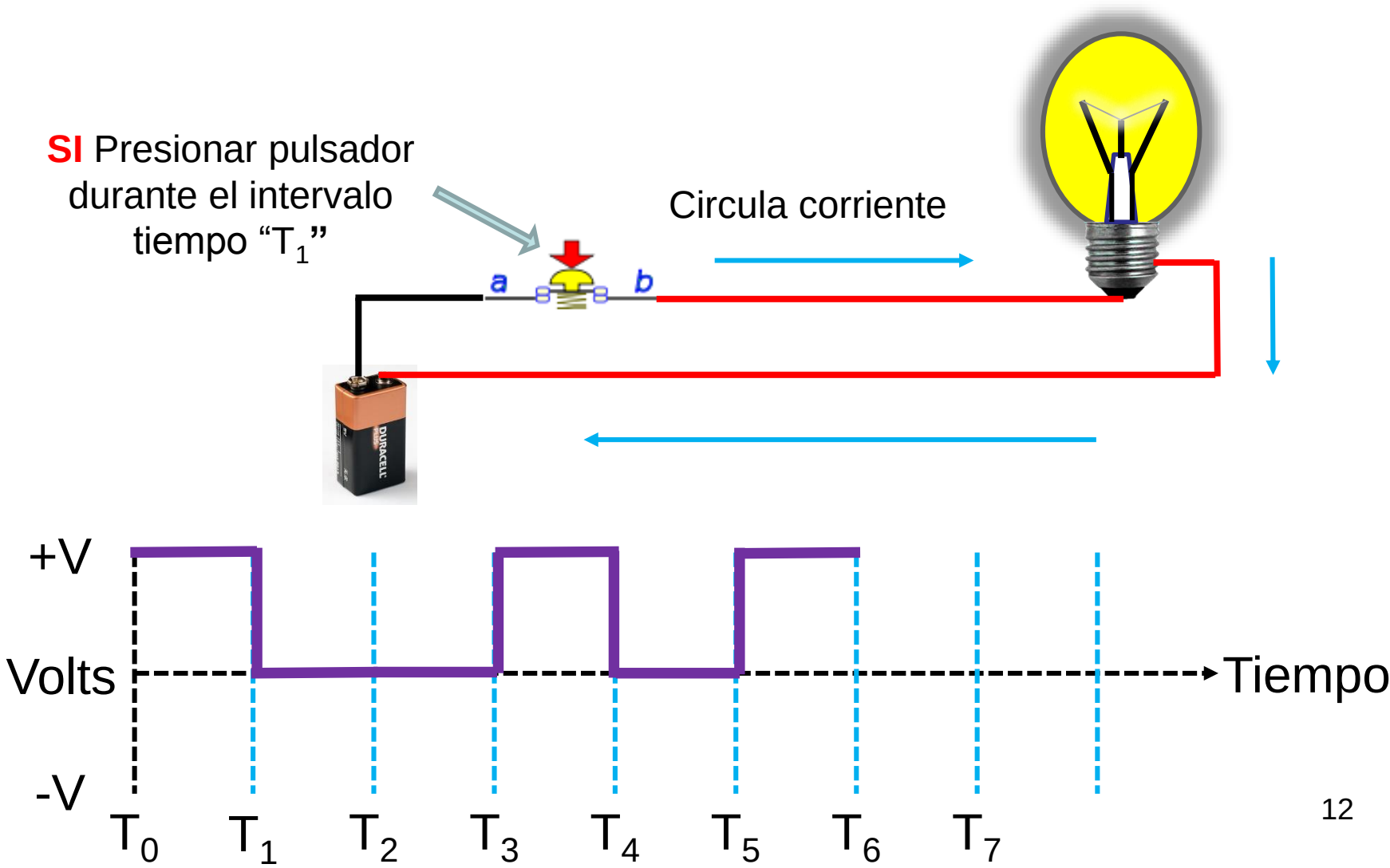
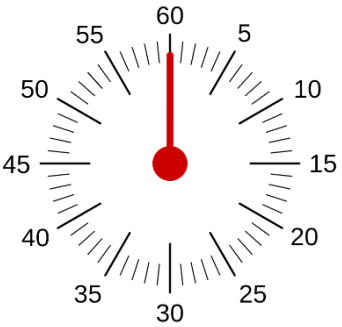


Sistemas de Representación

☐ ✓ ACCIÓN 6:

PULSAR SI / NO	PASO DE CORRIENTE
SI	✓
NO	X
NO	X
SI	✓
NO	X
SI	✓

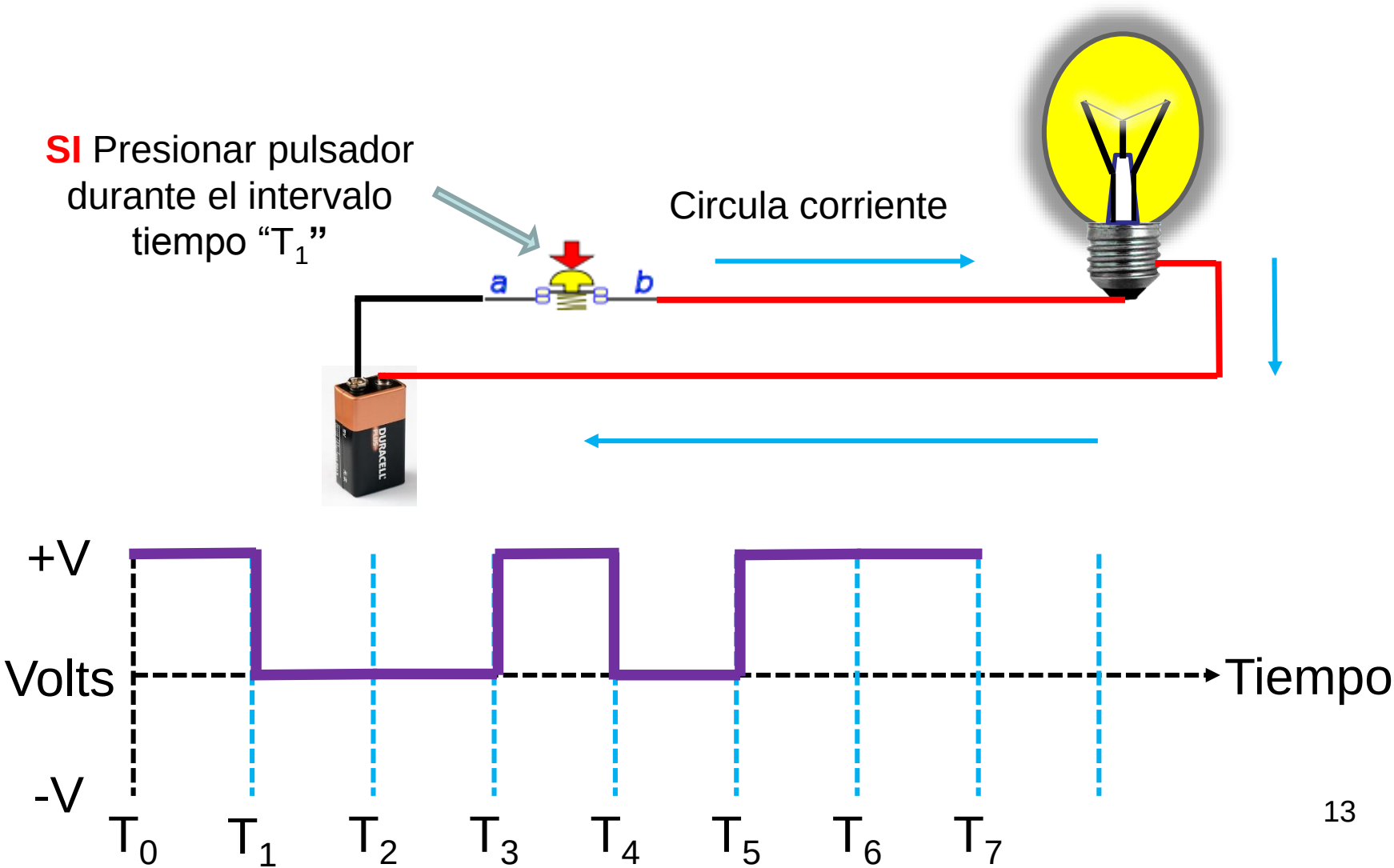
SUPONGAMOS:
 $0 - 60 = T_i$



Sistemas de Representación

☐ ✓ ACCIÓN 7:

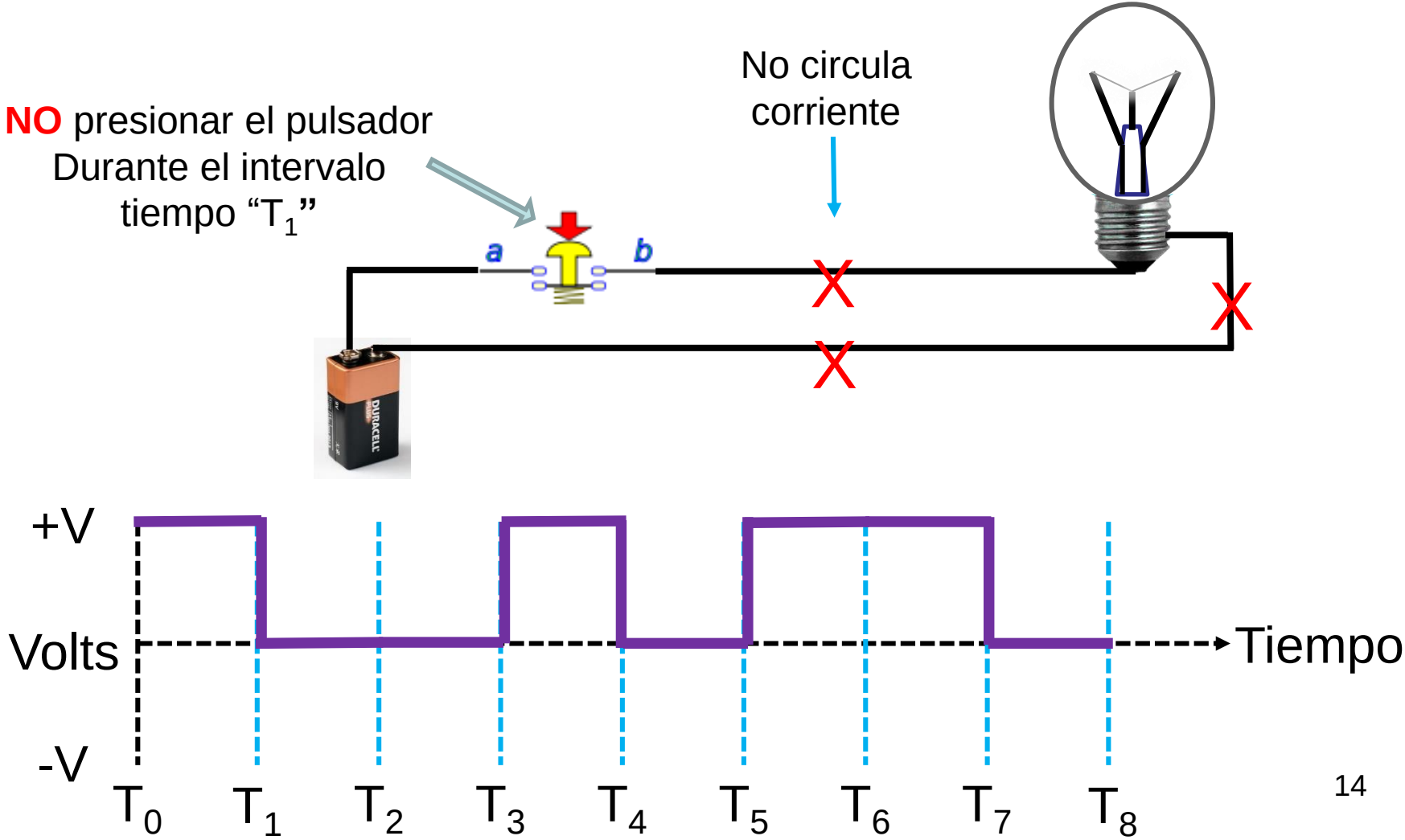
PULSAR SI / NO	PASO DE CORRIENTE
SI	✓
NO	X
NO	X
SI	✓
NO	X
SI	✓
SI	✓



Sistemas de Representación

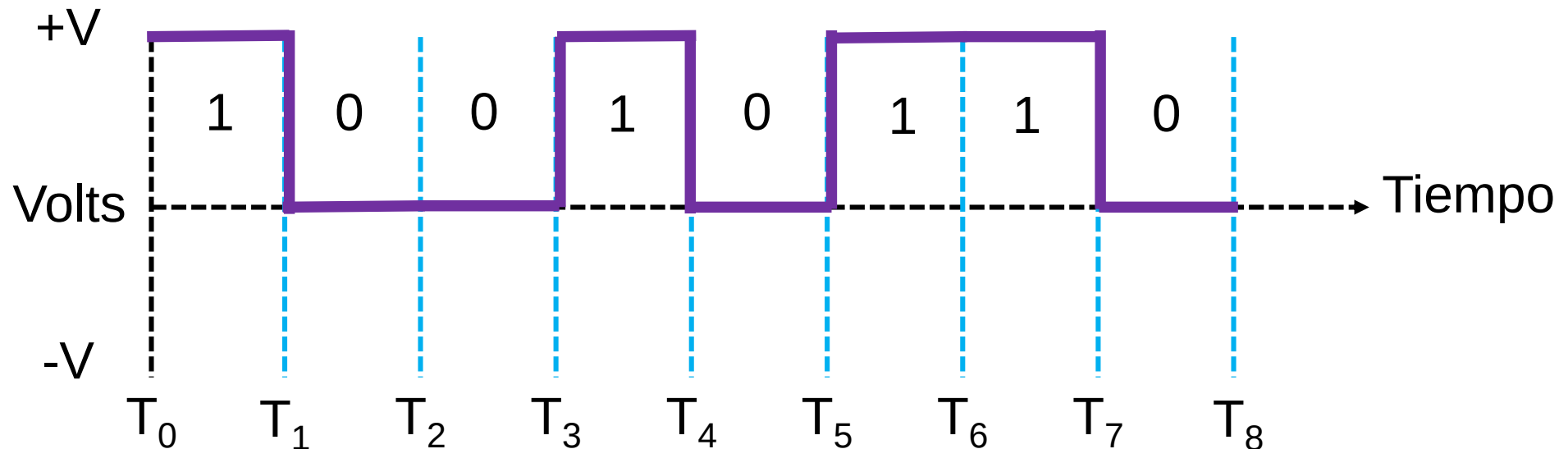
☐ ✓ ACCIÓN 8:

PULSAR SI / NO	PASO DE CORRIENTE
SI	✓
NO	X
NO	X
SI	✓
NO	X
SI	✓
SI	✓
NO	X



Sistemas de Representación

- ❑ ✓ EL MÉTODO DE TRANSMISIÓN, EN ESTE CASO, ES A TRAVÉS DE PULSOS ELÉCTRICOS QUE VARÍAN ENTRE DOS NIVELES DISTINTOS DE VOLTAJE:
- ❑ ✓ COMO EN ESTE CASO SE UTILIZAN DOS ESTADOS, APLICAMOS PARA SU REPRESENTACIÓN EL SISTEMA BINARIO, O SEA EL SISTEMA NUMÉRICO DE DOS DIGITOS :
 - **DIGITO “0” PARA INDICAR AUSENCIA DE LA CORRIENTE DURANTE UN INSTANTE T_i .**
 - **DIGITO “1” PARA INDICAR PRESENCIA DE LA CORRIENTE DURANTE UN INSTANTE T_i .**



Sistemas de Representación

- Las computadoras almacenan datos e instrucciones en memoria. Para ello utilizan el sistema binario.
- Razones:
 - ✓ El dispositivo se encuentra en uno de dos estados posibles (0 ó 1).
 - ✓ Identificar el estado es más fácil si sólo hay dos.
- Ejemplo:
 - ✓ Lámpara encendida o apagada.
 - ✓ Lámpara encendida con 10 intensidades distinta,
- Para saber si está encendida es más fácil en el primer caso que determinar alguna de las 10 intensidades distintas.

Unidades de medida para almacenamiento de datos

- Los computadores comprenden el lenguaje de los números
- La organización de un computador depende entre otros factores del sistema de representación numérica adoptado
- Se trabaja con el sistema binario, de donde proviene los términos: **Bit**: Acrónimo de **Binary Digit** (Dígito binario), es la mínima unidad de información que viaja y se almacena en un computador. Cada bit representa un impulso eléctrico (1 o 0).
- A un grupo de 8 Bits se lo denomina “BYTE” (y es el número de Bits necesarios para representar a un carácter) **“a” = 01100001**

Sistemas de Numeración – Introducción

- Los números naturales aparecen al contar los objetos de un conjunto.
- En la primera infancia se empieza por aprender a contar, luego más tarde se coordinan conjuntos prescindiendo del orden.
- Con posterioridad a la invención del número natural surge la necesidad de ampliar el concepto de número. Surgen así el **número entero**, el **número real**, etc.

Podemos definir un **Sistema de Numeración** como:

“El conjunto de reglas y convenciones que permiten la representación de todos los números mediante varios signos, o varias palabras”

Existen Sistemas de Numeración muy conocidos como el **Romano** que descompone el número en suma o diferencia de otros varios, cada uno de los cuales está representado por un símbolo especial:

I, V, X, L, C, D, M

Otro sistema es el **Decimal** que en vez de introducir nuevos símbolos para estos diversos sumandos, **utiliza el principio del valor relativo (posicional)**, es decir **una misma cifra representa valores distintos según el lugar que ocupa.**

Sistemas de Representación

- **Sistema Decimal:** Es el sistema de numeración utilizado en la vida cotidiana, cuya base es diez, utilizando los símbolos **0, 1, 2, 3, 4, 5, 6, 7, 8 y 9**
- **Sistema Binario:** los dos símbolos utilizados son el **0** y el **1**, los que reciben el nombre de **bit (Binary Digit)**
- **Sistema Octal:** de base 8, los símbolos utilizados son **0, 1, 2, 3, 4, 5, 6, 7**
- **Sistema Hexadecimal:** de base 16, los símbolos utilizados son **0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F**

Sistema Decimal: Inventado en la India en el siglo IV antes de Cristo y llevado a Europa por los árabes en la Edad Media.

Está fundado en el número fijo que llamamos **diez** (habiendo elegido este y no otro porque tenemos diez dedos en las manos).

De aquí surgen los **SISTEMAS NUMERICOS DE NOTACION POSICIONAL** donde:

NUMERO:

Representación de una cantidad, se compone de 1 o más dígitos

DÍGITO:

Elemento componente de un NUMERO para representar una cantidad

BASE:

Cantidad de símbolos diferentes que pueden representar **UN DÍGITO**

Ecuación General: $N = \sum_{i=-k}^{i=n} d_i * 10^i$ Donde 10 es la “Base”

Ej: $\begin{array}{|c|c|c|} \hline 2 & 1 & 2 \\ \hline \end{array})_d = 2*10*10 + 1*10 + 2*1$

$$= 2*10^2 + 1*10^1 + 2*10^0$$

Sistema de Numeración Binario

En el Sistema Binario los símbolos utilizados son el **0** y el **1**, llamados **dígitos binarios** (**bits**).

Es posible, aplicando el algoritmo de numeración, obtener los **números binarios** correspondientes a las primeras **16_{10}** (el subíndice **10** indica que el número esta en el **Sistema Decimal**) cantidades.

Igualmente podríamos obtener los números en el **Sistema Octal** y en el **Sistema Hexadecimal**.

DECIMAL b = 10 ₁₀	BINARIO b = 2 ₁₀	OCTAL b = 8 ₁₀	HEXADECIMAL b = 16 ₁₀
0	0	0	0
1	1	1	1
2	10	2	2
3	11	3	3
4	100	4	4
5	101	5	5
6	110	6	6
7	111	7	7
8	1000	10	8
9	1001	11	9
10	1010	12	A
11	1011	13	B
12	1100	14	C
13	1101	15	D
14	1110	16	E
15	1111	17	F
16	10000	20	10

Observemos que en el Sistema Hexadecimal de han agregado las letras **A, B, C, D, E y F** para obtener los 16 símbolos distintos necesarios.

La razón de tener en cuenta este Sistema es que $2^4 = 16$, por lo tanto a cada 4 cifras Binarias se corresponderá 1 cifra Hexadecimal.

También en el Sistema Octal se da algo parecido ya que $2^3 = 8$, por lo tanto cada tres cifras binarias se corresponde una Octal.

Esto último nos permite encontrar rápidamente las equivalencias entre Binario, Octal y Hexadecimal, por ejemplo:

$$\underbrace{0010}_2 \underbrace{1001}_9 \underbrace{1110}_E \underbrace{0101}_5)_2 = 29E5)_{16}$$

$$\underbrace{110}_6 \underbrace{111}_7 \underbrace{111}_7 \underbrace{001}_1 \underbrace{000}_0 \underbrace{110}_6)_2 = 677106)_8$$

Conversión entre números de distintas bases

Se subdivide en dos etapas:

a-) Por un lado la **parte entera**

b-) Por otro lado la **parte decimal**

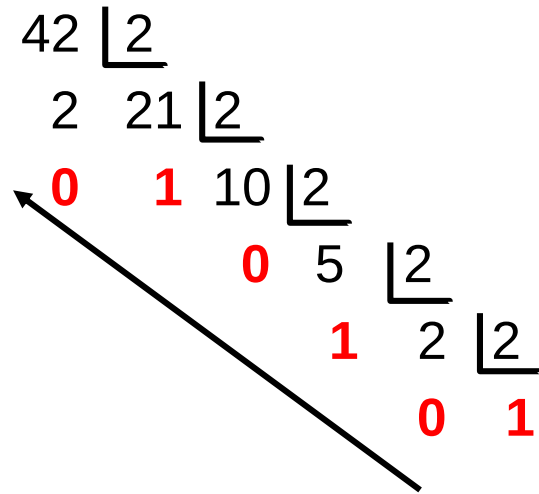
a-) Se divide el número por la nueva Base. El **cociente** se escribe debajo del número original y repetimos el procedimiento hasta que el **residuo** sea menor que **divisor**.

El nuevo número se forma de derecha a izquierda a partir del último cociente tomando todos los residuos de las divisiones anteriores.

b-) La parte fraccionaria se convierte de la siguiente manera: Multiplicamos el número por la Base, tomamos la parte entera del resultado y la parte fraccionaria la volvemos a multiplicar sucesivamente hasta llegar a cero.

Ej: Pasar el número $42,375)_{10}$ a “Binario”:

a) Parte Entera:



b) Parte Fraccionaria:

$$\begin{array}{l} 0,375 * 2 = 0,75 \\ 0,75 * 2 = 1,50 \\ 0,50 * 2 = 1,00 \end{array} \left| \begin{array}{l} \\ \\ \downarrow \end{array} \right.$$

$$42,375)_{10} = 101010,011)_2$$

Conversión a Base Decimal

Tomamos los dígitos y los multiplicamos por la base elevada a la posición que ocupa cada dígito:

Ej: Pasar el número $204)_8$ a “**Decimal**”:

$$\begin{aligned} 204)_8 &= 2 * 8^2 + 0 * 8^1 + 4 * 8^0 = \\ &= 2 * 64 + 0 + 4 * 1 = \\ &= 128 + 0 + 4 = 132)_{10} \end{aligned}$$

Ejercicio:

$$10101)_2 = ???$$

Pasaje de una Base a otra en forma directa

El pasaje de 2 a 8 ó 16, o el pasaje en sentido contrario, se puede realizar en forma mecánica, sin necesidad de realizar los pasos anteriores.

Cuando una base es potencia de otra, se reemplaza cada dígito por su equivalente numérico de la segunda base.

Sistemas Potencia de “2”: 2^n

$n = 1$	$\rightarrow$	$2^1 = 2$	$\rightarrow$	(BINARIO)	cada dígito en la base se representa con “n” dígitos binarios
$n = 3$	$\rightarrow$	$2^3 = 8$	$\rightarrow$	(OCTAL)	
$n = 4$	$\rightarrow$	$2^4 = 16$	$\rightarrow$	(HEXADECIMAL)	

Conversión de Binario a Octal por Agrupación

El pasaje de 2 a 8 o 16, o el pasaje en sentido contrario, se puede realizar en forma mecánica, sin necesidad de realizar ningún cálculo.-

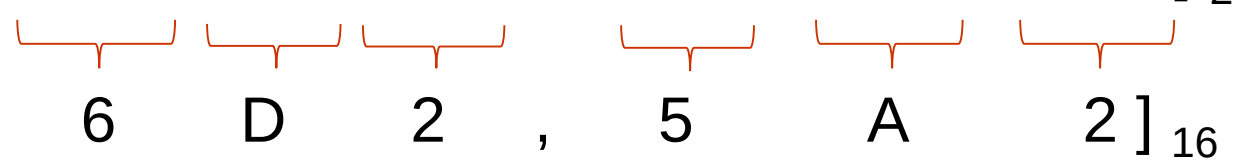
Cuando una base es potencia de otra (ya sea en forma positiva o en forma negativa) por ejemplo lo son las bases 8 y 2 ($8 = 2^3$) *se reemplaza cada dígito por su equivalente numérico de la segunda*

Demostración para el pasaje de base 2 a 8

$$N)_b = 110\ 101, 110)_2 =$$

$$\begin{aligned} &= 1x2^5 + 1x2^4 + 0x2^3 \quad + \quad 1x2^2 + 0x2^1 + 1x2^0 \quad + \quad 1x2^{-1} + 1x2^{-2} + 0x2^{-3} \\ &= [1x2^2 + 1x2^1 + 0x2^0]2^3 + [1x2^2 + 0x2^1 + 1x2^0]2^0 + [1x2^2 + 1x2^1 + 0x2^0]2^{-3} \\ &= [4 + 2 + 0]2^3 + [4 + 0 + 1]2^0 + [4 + 2 + 0]2^{-3} \\ &= \quad \quad 6 \times 8^1 \quad + \quad \quad \quad 5 \times 8^0 \quad + \quad \quad 6 \times 8^{-1} \\ &= \quad \quad \quad 6 \quad 5 \quad , \quad 6)_8 \end{aligned}$$

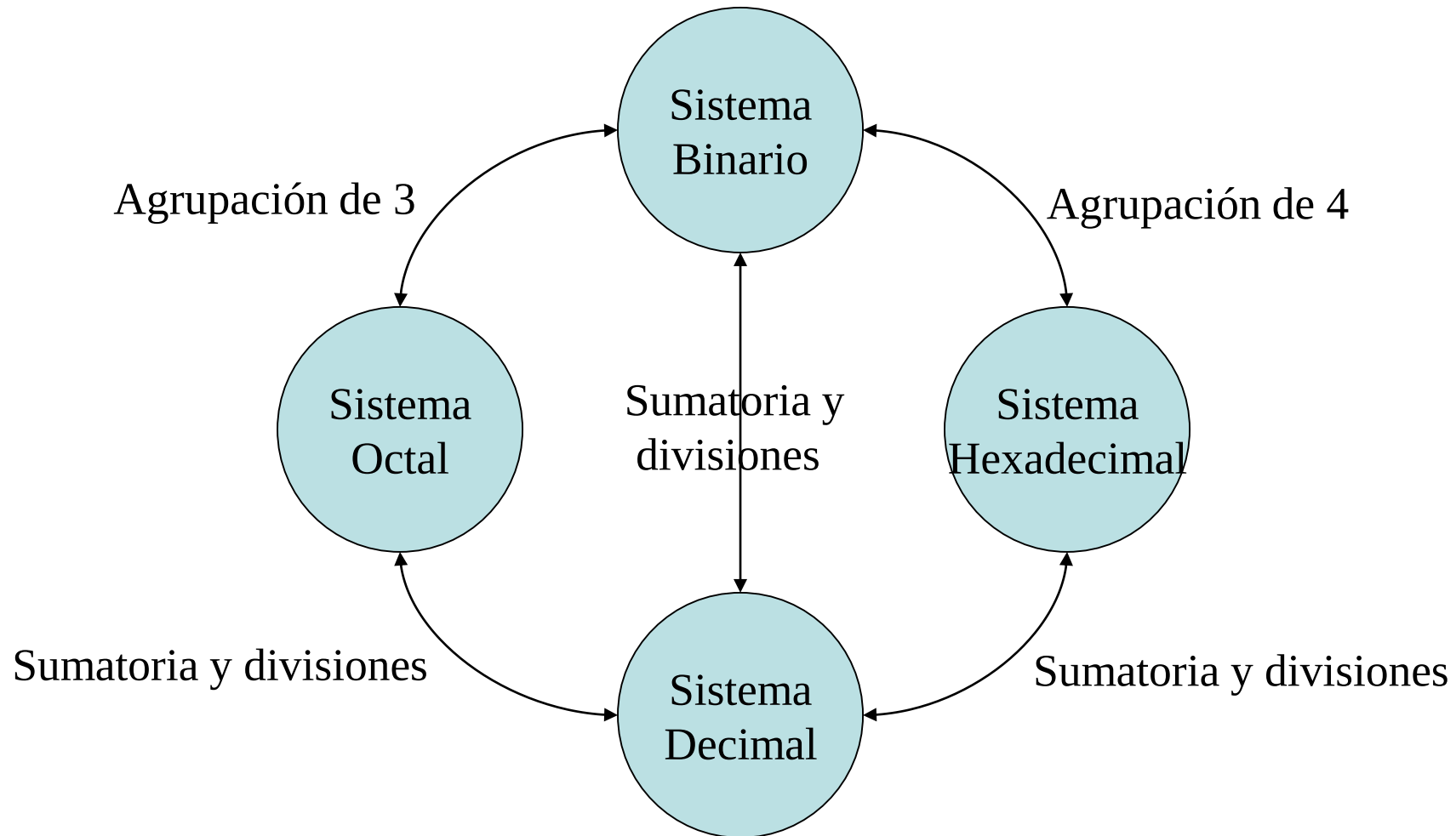
Conversión de Binario a Hexadecimal Por agrupación

$\textcolor{red}{0}110\ 1101\ 0010\ ,\ 0101\ 1010\ 0010\]_2$
 $6\ D\ 2\ ,\ 5\ A\ 2\]_{16}$

Conversión de Hexadecimal a Binario Por agrupación

$A\ 2\ ,\ 1\ 7\]_{16}$
 $1010\ 0010\ ,\ 0001\ 0111\]_2$

Conversión de bases



Ejercicios

- Pasar a **Binario**

$$204)_8 = ???$$

$$A3E5)_{16} = ???$$

- Pasar a **Octal**

$$1F4)_{16} = ???$$

$$101010)_2 = ???$$

- Pasar a **Hexadecimal**

$$4275)_8 = ???$$

$$11110010)_2 = ???$$

Aritmética binaria

- Una computadora digital consume la mayor parte de su vida útil a la realización de operaciones de aritmética binaria.
- Las reglas de la aritmética binaria son similares a las de la aritmética decimal. Los conceptos de acarreo y préstamo, por ejemplo, se aplican también a la aritmética binaria.
- Todas las operaciones aritméticas estudiadas en el álgebra aplicadas a números al **Sistema Decimal**, son aplicables a los **Números Binarios** dado que ambos Sistemas responden al mismo algoritmo de numeración.

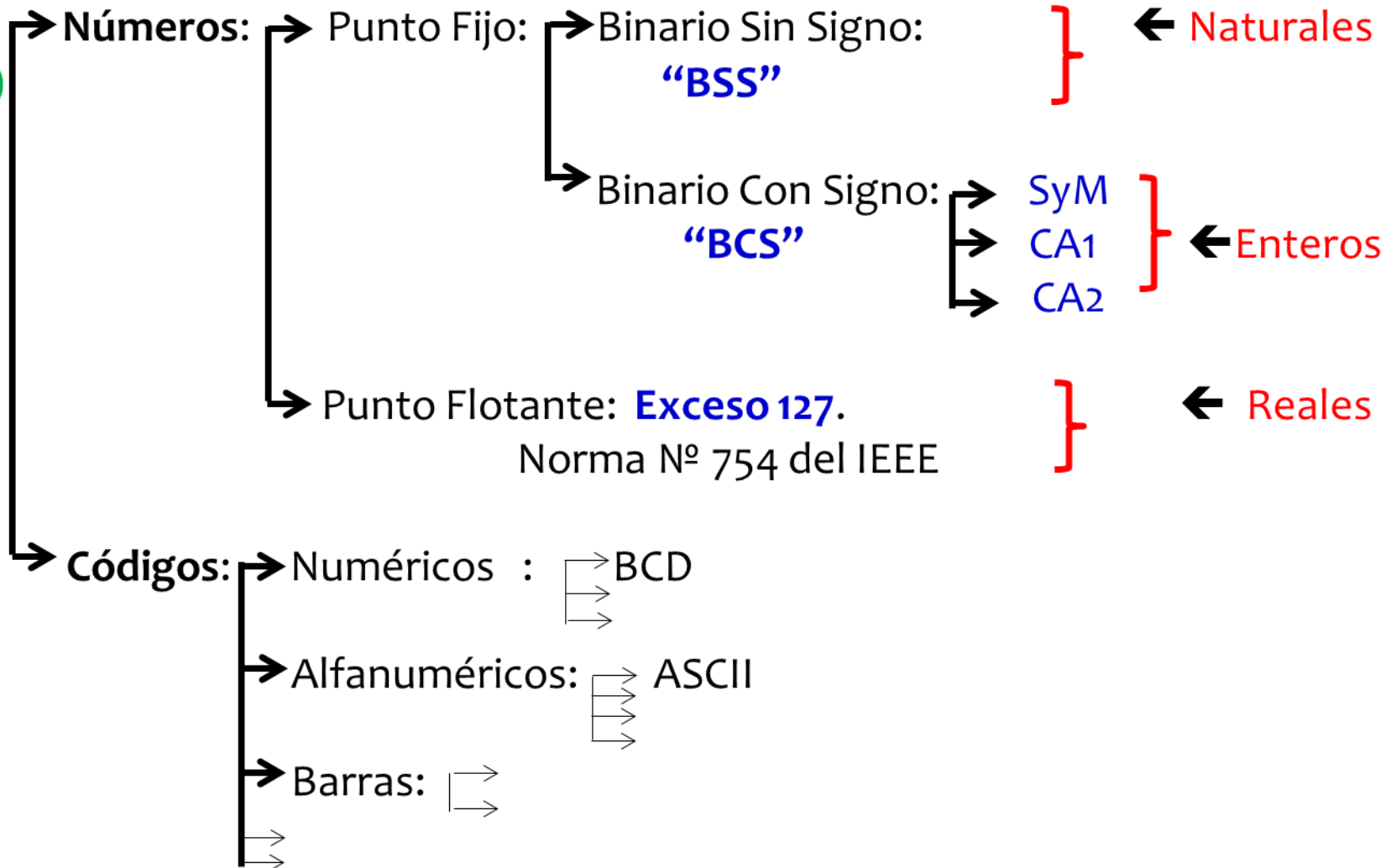
Operaciones de aritmética binaria

Todas las operaciones aritméticas estudiadas en el álgebra aplicadas a números al **Sistema Decimal**, son aplicables a los **Números Binarios** dado que ambos Sistemas responden al mismo algoritmo de numeración.

Dígitos		Suma		Resta		Multiplicación
a_1	a_2	$a_1 + a_2$		$a_1 - a_2$		$a_1 \cdot a_2$
		Suma	Acarreo	Diferencia	Préstamo	Producto
0	0	0	0	0	0	0
0	1	1	0	1	1	0
1	0	1	0	1	0	0
1	1	0	1	0	0	1

Representación de

Datos:
("0 y 1")



Números sin Signo (BSS)

En binario para “n” cantidad de bits podemos expresar esto:

Número mínimo = 0

Número máximo = $2^n - 1$

Rango = 2^n

En Binario y Sin Signo (BSS) según la cantidad de bits tenemos

Con 4 bits puedo representar 16 números y el 15 es el mayor.

Con 8 bits puedo representar 256 números y el 255 es el mayor.

Con 10 bits puedo representar 1.024 números y el 1.023 es el mayor.

Con 12 bits puedo representar 4.096 números y el 4.095 es el mayor.

Con 16 bits puedo representar 65.536 números y el 65.535 es el mayor.

Con 32 bits puedo representar 4.294.967.296 números y el 4.294.967.295 es el mayor.

Con 64 bits puedo representar 18.446.744.073.709.551.616 números

Representación de Números con Signo (BCS)

- Con n bits, 1 bit representa al signo y $n-1$ bits a la magnitud

b_{n-1}	b_{n-2}	b_0
Signo	Magnitud	

- El bit $n-1$ (extremo izquierdo) representa sólo al signo
- Los bits 0 a $n-2$ la magnitud
- ✓ Un **0** en el bit de signo indica que el número es positivo
- ✓ Un **1** en el bit de signo indica que el número es negativo
- ✓ Los bits **0** a **$n-2$** representan el valor absoluto en binario
- ✓ El rango: depende de la convención (**SyM**, **Ca2** o **Ca1**)

Representación de Números con Signo (BCS)

En matemáticas, los números negativos en cualquier base se representan del modo habitual (**precediéndolos con el símbolo “-”**)

Sin embargo, en una computadora existen varias formas de representar el signo de un número:

- **Signo y Módulo**
- **Complemento a la Base**
- **Complemento a la Base-1**
- **Exceso**

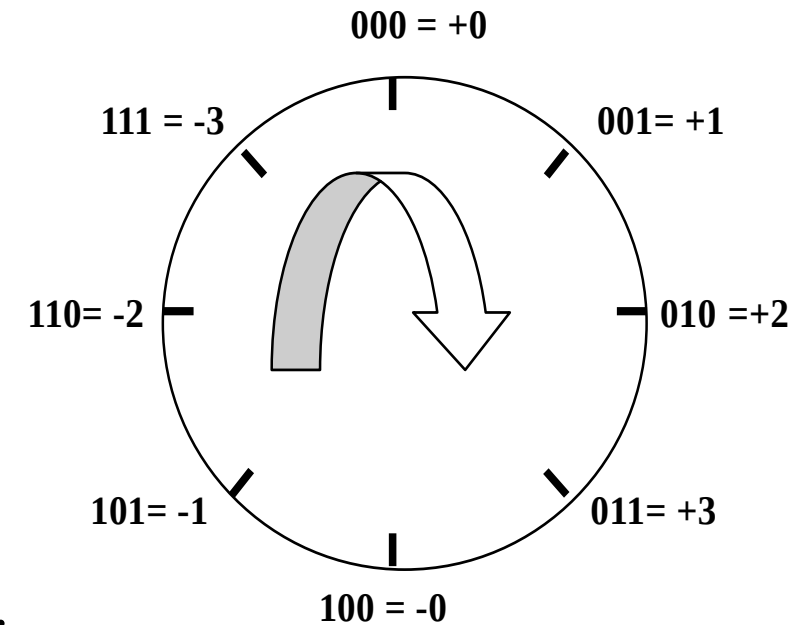
Representación en Signo y Módulo

Para representar un número signado de “n bits” se realiza lo siguiente:

1- Asignar un bit para representar el signo (**el MSB - Most Significant Bit o Bit Más Significativo**) y por convención:

- Si es igual a “0”: indica un número **positivo**.
- Si es igual a “1”: indica un número **negativo**.

2- Los “n-1 bits” restantes se utilizan para representar el significado, que es la magnitud del número en valor absoluto.



Este enfoque es directamente comparable a la forma habitual de mostrar el signo (colocando “+” o “-” al lado de la magnitud del número).

Ejemplo de Signo y Módulo

Sea una representación que nos permite codificar un número entero en binario con 8 bits (un byte)

Esto nos otorga **1 bit para el signo** y **7 bits para la magnitud**.

8 bits permiten representar en teoría al menos $2^8 = 256$ números.

128 números positivos (Bit de Signo en “0”)

128 números negativos (Bit de Signo en “1”)

Convención Signo y Modulo

➤ Ejemplo:

✓ $+5)_{10} = 0000\ 0101)_2$

Diagram illustrating the binary representation of +5. A green box contains the weights 64, 32, 16, 8, 4, 2. Green arrows point from the 8, 4, and 2 weights to the 1s in the binary digits 0101. A blue arrow points from the 5 in the decimal expression to the 0101 part of the binary result. A bracket under the 0101 part is labeled 5.

✓ $+25)_{10} = 0001\ 1001)_2$

Diagram illustrating the binary representation of +25. A green box contains the weights 64, 32, 16, 8, 4, 2, 1. Green arrows point from the 16, 8, and 1 weights to the 1s in the binary digits 1001. A blue arrow points from the 25 in the decimal expression to the 1001 part of the binary result. A bracket under the 1001 part is labeled 25.

$-5)_{10} = 1000\ 0101)_2$

Diagram illustrating the binary representation of -5. A blue arrow points from the 5 in the decimal expression to the 0101 part of the binary result. A bracket under the 0101 part is labeled 5.

$-25)_{10} = 1001\ 1001)_2$

Diagram illustrating the binary representation of -25. A blue arrow points from the 25 in the decimal expression to the 1001 part of the binary result. A bracket under the 1001 part is labeled 25.

Tabla para n=8 (8 bits) en Signo y Módulo

Valores de 8 Bits	Interpretado en Signo y Módulo en Decimal	Interpretado como entero sin Signo y Módulo
00000000	0	0
00000001	1	1
00000010	2	2
00000011	3	3
.....		
01111110	126	126
01111111	127	127
10000000	-0	128
10000001	-1	129
10000010	-2	130
10000011	-3	131
.....		
11111101	-125	253
11111110	-126	254
11111111	-127	255

Ejemplo: Representar el número $-97)_{10}$

Pasos:

- 1- Como el número -97 es negativo $\Rightarrow$ deberá llevar como Bit de Signo "1"
- 2- Realizar la conversión en Valor Absoluto de $-97)_{10} \Rightarrow |-97)_{10}| = 97)_{10}$
Que en Binario corresponde al número $1100001)_2$
- 3- Armar el número, el número $-97)_{10}$ en binario con formato de Signo y Módulo es :

 $11100001)_2 \Rightarrow$ Donde el **1** en el **MSB** indica un número **negativo**, y

 $1100001)_2$ es el significado en **Valor Absoluto**.

Desventajas de la Representación en Signo y Módulo

(Siguiendo el ejemplo de un número de 8 bits)

- * Es más complejo operar aritméticamente.

Para realizar una suma por ejemplo: Primero hay que determinar si los dos números poseen el mismo signo, y en caso de ser así, realizar la suma de la parte significativa, pero en caso contrario, restar el mayor del menor y asignar el signo del mayor.

- * Posee la doble representación del Cero

Aparece el cero signado:

00000000₂ (+0₁₀)

10000000₂ (-0₁₀)

Ventajas de la Representación en Signo y Módulo

(Siguiendo el ejemplo de un número de 8 bits)

* Posee un rango simétrico.

Los número van desde el $+127_{10} = 01111111_2$, Pasando por el $+0_{10} = 00000000_2$ y el $-0_{10} = 10000000_2$, hasta el $-127_{10} = 11111111_2$

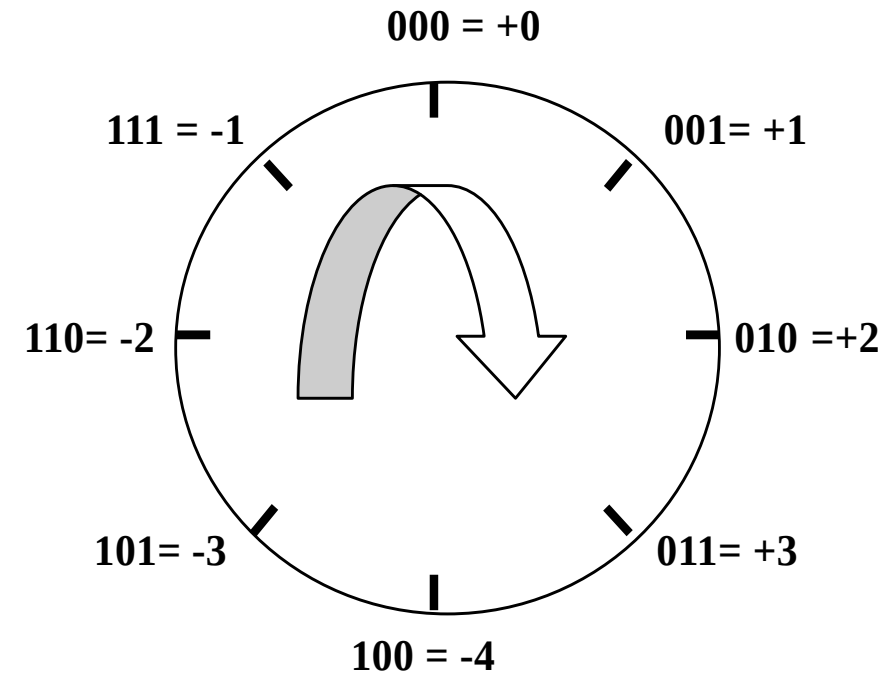
En forma general:

Para “**n bits**”, el rango (en decimal) para Signo y Módulo es:

$$(-2^{n-1} - 1; 2^{n-1} - 1) \text{ o bien } +/- 2^{n-1} - 1$$

Complemento a la Base (C_{a2})

- El complemento a la base de un número N , es lo que lo falta a N para llegar a el modulo 10^n .
- Los números positivos comienzan con cero (0)
- Los negativos comienzan con uno (1)
- El rango es asimétrico y va desde $-(2^{n-1})$ a $+(2^{n-1}-1)$
- Existe una sola representación del cero (0)



Expresión General: $N + C_B = 10^n \rightarrow C_B = 10^n - N$

Complemento a la Base (C_{a2})

$$N + C_B = 10^n \Rightarrow C_B = 10^n - N$$

$$C_{a2} = 2^n - N$$

➤ Ejemplo:

$4 + 1 = 5$
 64 32 16 8 4 2 1
 ↑ ↑ ↑ ↑ ↑ ↑ ↑

✓ $+5)_{10} = 0000\ 0101)_2$ $-5)_{10} = 1111\ 1011)_2$

Regla práctica:

Tomamos el numero positivo y de derecha a izquierda se copia hasta el primer "1" y luego invertimos todos los demás, 0 por 1 y 1 por 0.

Esto se puede realizar para pasar de Nro. positivo al negativo y también para pasar de negativo al positivo

64 32 16 8 4 2 1
 ↓ ↓ ↓ ↓ ↓ ↓ ↓
 16 + 8 + 1 = 25

✓ $+25)_{10} = 0001\ 1001)_2$ $-25)_{10} = 1110\ 0111)_2$

Ejemplo (en Binario):

Dado $+2_{10}$ expresar su C_2 con 3 bits:

$$+2_{10} = 010_2$$

$$2^n = 2^3 = 1000_2$$

$$C_n = 10^n - N$$

$$C_2 = 2^3 - N = 2^3 - 2 = 1000_2 - 010_2$$



$$\begin{array}{r} 1000_2 \\ - 010_2 \\ \hline 110_2 \end{array}$$

Regla de Nemotécnica:

$$C_b = C_{b-1} + 1$$

Operaciones en Complemento a 2

Suma en Complemento a 2

- Los dos números en una suma se denominan **sumandos**.
- El resultado es la suma de ambos.
- Cuando se suman dos números binarios con signo pueden producirse cuatro casos:
 1. **Ambos números son positivos.**
 2. **El número positivo es mayor que el negativo en valor absoluto.**
 3. **El número negativo es mayor que el positivo en valor absoluto.**
 4. **Ambos números son negativos.**

Operaciones en Complemento a 2

Suma en Complemento a 2

- Ambos números son positivos:

$$\begin{array}{r} 00000111 \\ + 00000100 \\ \hline 00001011 \end{array} \quad \begin{array}{r} 7 \\ + 4 \\ \hline 11 \end{array}$$

- La suma es positiva y, por tanto, es un número binario real (no complementado).

Operaciones en Complemento a 2

Suma en Complemento a 2

- El número positivo es mayor que el número negativo en valor absoluto:

$$\begin{array}{r} 00001111 \quad 15 \\ + 11111010 \quad -6 \\ \hline 1\ 00001001 \quad 9 \end{array}$$

Acarreo que se descarta →

- El bit de acarreo final no se tiene en cuenta. La suma es positiva y, por tanto es un número binario real (no complementado).

Operaciones en Complemento a 2

Suma en Complemento a 2

- El número negativo es mayor que el número positivo en valor absoluto:

$$\begin{array}{r} 00010000 \\ + 11101000 \\ \hline 11111000 \end{array} \quad \begin{array}{r} 16 \\ + -24 \\ \hline -8 \end{array}$$

- La suma es negativa y, por tanto, está en complemento a 2.

Operaciones en Complemento a 2

Suma en Complemento a 2

- Ambos números son negativos:

$$\begin{array}{r} 11111011 \quad -5 \\ + 11110111 \quad + -9 \\ \hline 111110010 \quad -14 \end{array}$$

Acarreo que se descarta → 1

- El bit de acarreo final no se tiene en cuenta. La suma es negativa y, por tanto, está en complemento a 2.

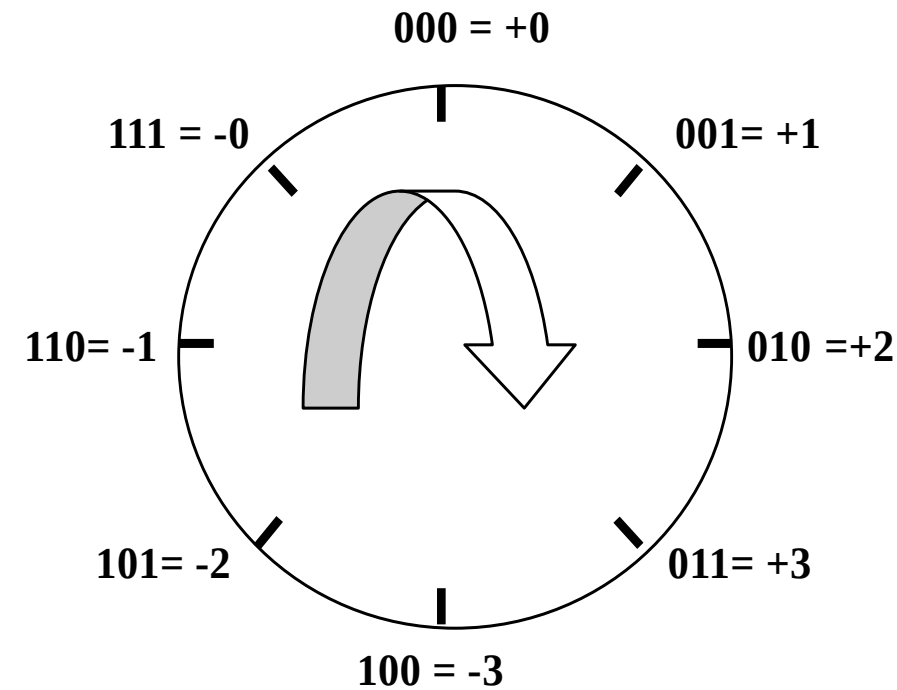
Al no restar el “1” como en el método anterior se logra lo siguiente:

- Existe una sola representación del cero.
- Se representa un número negativo más al no tener el “-0”.

En lugar del “-0” ahora podemos representar el “-4”, por lo tanto, el máximo número positivo a representar con “n” bits es $+2^{(n-1)}-1$, y máximo número negativo a representar con “n” bits es $-2^{(n-1)}$

Complemento a la Base-1 (C_{a1})

- El **complemento a la base menos uno** de un número N , es lo que lo falta a N para llegar a el modulo 10^{n-1}
 - En **Base 2** se da una regla práctica en donde el C_{a1} de un número se obtiene invirtiendo todos los bits
 - Si el número es positivo, los n bits poseen la representación binaria del número (como siempre)
 - Si el número es negativo, los n bits poseen el C_{a1} del valor deseado.
-
- ✓ Los n bits representan al número.
 - ✓ Los positivos empiezan con cero (**0**)
 - ✓ Los negativos empiezan con uno (**1**)
 - ✓ El rango: $-(2n-1-1)$ a $+(2n-1-1)$ con doble representación del cero (**+0** y **-0**)



Expresión General:

$$N + C_{B-1} = 10^n - 1 \rightarrow$$

"n= Cantidad de Bits"

$$C_{B-1} = (10^n - 1) - N$$

Complemento a la Base-1 (C_{a1})

$$C_{B-1} = (10^n - 1) - N_{\text{º}}$$

$$C_{1a} = (2^n - 1) - N_{\text{º}}$$

➤ Ejemplo:

4 + 1 = 5

64	32	16	8	4	2	1
↑	↑	↑	↑	↑	↑	↑

✓ $+5)_{10} = \mathbf{0000\ 0101}_2$ $-5)_{10} = \mathbf{1111\ 1010}_2$

Regla práctica:

Se invierten todos los bits (0 por 1 y 1 por 0).

Esto se puede realizar para pasar de Nro. positivo al negativo y también para pasar de negativo al positivo

✓ $+25)_{10} = \mathbf{0001\ 1001}_2$ $-25)_{10} = \mathbf{1110\ 0110}_2$

64	32	16	8	4	2	1
↓	↓	↓	↓	↓	↓	↓

16 + 8 + 1 = 25

Ejemplo (en Binario):

(C_{a1})

Dado $+2_{10}$ expresar su C_{a1} con 3 bits:

$$+2_{10} = 010_2 \quad (\text{con 3 bits})$$

$$2^n = 2^3 = 1000_2$$

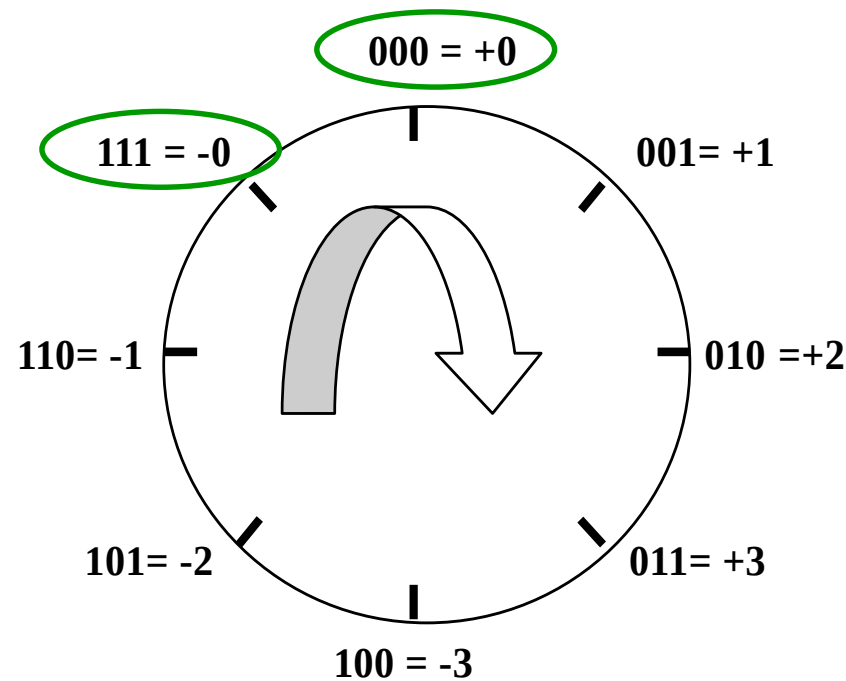
$$C_{a1} = (10^n - 1) - N$$

$$2^n - 1 = 2^3 - 1 = 1000_2 - 1_2 = 111_2$$

$$C_1 = 2^3 - 1 - 2 = 111_2 - 010_2$$

$$\begin{array}{r} 111_2 \\ - 010_2 \\ \hline 101_2 \end{array} \Rightarrow C_1 \text{ de } +2_{10}$$

(C_{a1})



- Se puede observar la “**Doble Representación del Cero**” (**000** y **111**), además de una propiedad particular por la disposición de los números que es la Siguiente:
- Dado un número **POSITIVO** o **NEGATIVO** de “n” bits, su complemento tiene los “n” bits todos invertidos. Ej.: $+1_{10} = 001_2$ y $-1_{10} = 110_2$
- **Regla de Conversión:**

**DADO EL NÚMERO “N”, SU COMPLEMENTO A LA BASE MENOS 1
POSEE TODOS SUS BITS INVERTIDOS.**

Complemento a la Base-1 (C_{a1})

$$C_{B-1} = (10^n - 1) - N$$

➤ Ejemplo:

Dada una cadena de bits ¿qué número decimal representa si lo interpretamos en C_{a1} ?

✓ Cuando es positivo:

$$\mathbf{0}000\ 0101 = 1 \times 2^2 + 1 \times 2^0 = 4 + 1 = +5$$

✓ Cuando es negativo, saco el C_{a1} del número y obtengo el positivo y luego cambio el signo

$$\mathbf{1}111\ 1010 \rightarrow 0000\ 0101 = +5_{10}$$

$$1111\ 1010 = -5_{10}$$

Operaciones en Complemento a 1

Resta o Sustracción en Complemento a 1

- La sustracción es un caso especial de la suma.
- Por ejemplo, restar +6 (el **sustraendo**) de +9 (el **minuendo**) es equivalente a sumar -6 a +9.
- Básicamente la operación de la sustracción cambia el signo del sustraendo y le suma al minuendo.
- El resultado de una sustracción se denomina **diferencia**.
 - El signo de un número binario positivo o negativo se cambia calculando su complemento a 1.
 - Para restar dos números con signo se calcula el complemento a 1 del sustraendo y se suman. Cualquier bit de acarreo final se suma al LSB (de más a la derecha).

Operaciones en Complemento a 1

Resta o Sustracción en Complemento a 1

- Ejemplo:

Forma normal		Forma en complemento a 1	
00111111		00111 111	63_{10}
- 00011100		+ 11100011	- 28_{10}
		100100010	
Acarreo final	→		
		1	
		00100011	
			35_{10}

- Si no hubiera un bit de acarreo final, entonces el resultado es un número negativo representado en la forma de complemento a 1.

Operaciones en Complemento a 1

Resta o Sustracción en Complemento a 1

- **Ejemplo:** Restar 63_{10} de 28_{10} .

Forma normal		Forma en complemento a 1
00011100		00011100 28_{10}
- 00111111	+	<u>11000000</u> - 63_{10}
No hay acarreo final →		
		11011100 - 35_{10}

- No se tiene un bit de acarreo final, por tanto el resultado es un número negativo en complemento a 1.
- Se debe determinar su complemento a 1 para obtener su magnitud; en este caso es: 00100011 o 35_{10} .
- Como su signo es negativo, el resultado real es: -35_{10} .

Representación de números sin/con signo, para 3 bits

BSS			BCS								
			S y M			Ca1			Ca2		
7	111	2^{n-1}									
6	110					$C_{a1} = 10^n - 1 - N_0$			$C_{a2} = 10^n - N_0$		
5	101					Se invierten todos los bits			De derecha a Izq. se copia hasta el 1ªbits luego se invierten todos		
4	100										
3	011		+3	011	$+ (2^{n-1} - 1)$	+3	011	$+ (2^{n-1} - 1)$	+3	011	$+ (2^{n-1} - 1)$
2	010		+2	010		+2	010		+2	010	
1	001		+1	001		+1	001		+1	001	
0	000	0	0	000	0	0	000	0	0	000	0
			-1	101		-1	110		-1	111	
			-2	110		-2	101		-2	110	
			-3	111	$- (2^{n-1} - 1)$	-3	100	$- (2^{n-1} - 1)$	-3	101	
									-4	100	$- (2^{n-1})$

Representación en Punto Flotante

- Es una convención utilizada para trabajar con números Reales de alto rango, o sea, para poder trabajar con números de mayor cantidad de bits de los que posee la PALABRA de la máquina.
- Los números Naturales y Enteros que vimos hasta ahora se denominan “**Punto Fijo**” o “**Coma Fija**”, y si por ejemplo nuestra computadora puede operar solamente con 3 dígitos, podríamos trabajar con 1000 combinaciones.
- Sería como una representación exponencial de los números decimales Convencional, semejante a la notación científica.

Ej. : $12,3 \times 10^{+6}]_d$ o $3,25 \times 10^{-8}]_d$ $\Rightarrow$

$$N_d = \pm m * 10^{\pm P}$$

Punto Flotante

Exceso 127 IEEE Normalizado

- Utiliza para la escritura Base 2 para las fracciones y notación en exceso para los exponentes.
- La norma define tres formatos: Precisión Sencilla (32 bits), Precisión Doble (64 bits) y Precisión Extendida (80 bits).
- Utilizaremos el formato de **Precisión Sencilla (32 bits)**:
 - ✓ Se inicia con **1 bit de signo**, (0 = Positivos y 1 = Negativos)
 - ✓ **8 bits para el exponente** (desplazado en 127)
 - ✓ **23 bits para la mantisa**

Posee el siguiente formato:

- Partiendo del $N^{\circ} = +/- \mathbf{m} * \mathbf{b}^c$
- Donde “**m**”: Mantisa y “**b**”: La base en la cual se encuentra expresado el nro.
- La característica $\mathbf{C} = 127 +/- \mathbf{p}$
- La normalización se realiza de la siguiente forma: $\mathbf{m} = 1, \mathbf{F} = 1, \mathbf{X}_1\mathbf{X}_2\mathbf{X}_3\dots\mathbf{X}_n$

Reglas básicas para la norma:

- ✓ La representación se realiza en binario y de la forma: $+/- 1, \mathbf{F} * 10_2^{+/-p}$ donde $\mathbf{F} = \mathbf{X}_1\mathbf{X}_2\mathbf{X}_3\dots\mathbf{X}_n$
- ✓ En precisión simple se utilizan **32 bits (4bytes)**
- ✓ La parte fraccionaria se representa en los últimos **23 bits**, sobreentendiéndose que posee un “**1**” en la parte entera, que no se representa en su normalización.
- ✓ El primer bit del primer byte dará el signo de la mantisa, tratándose de representar la fracción como un número del tipo Signo y Modulo.
- ✓ Al exponente se lo desplaza **127**, dando la característica “**+/- p + 127**” y se anotan en los **8 bits** siguientes al signo.
- ✓ El número “0” se representa como “+0”.

Formato:

S	CARACTERISTICA	MANTISA
S	E E E E E E E E	X ₁ X ₂ X ₃ X _n
1 bit	8 bits	23 bits

Total= 1 + 8 + 23 = 32 bits

Ej:

$5)_{10} = 101)_2 = +1,01 * 10_2$ **10** => Se desplazó la coma **2** lugares a la izquierda para normalizar



$$C = 127 +/- p = 127 + 2 = 129)_{10} = 10000001)_2$$

S	CARACTERISTICA	MANTISA
0	1 0 0 0 0 0 0 1	0 1 0

Exceso 127 IEEE Normalizado

$$N_o = \pm m 10^{\pm e}$$

Signo de mantisa → +/- = 0/1

Polarizado

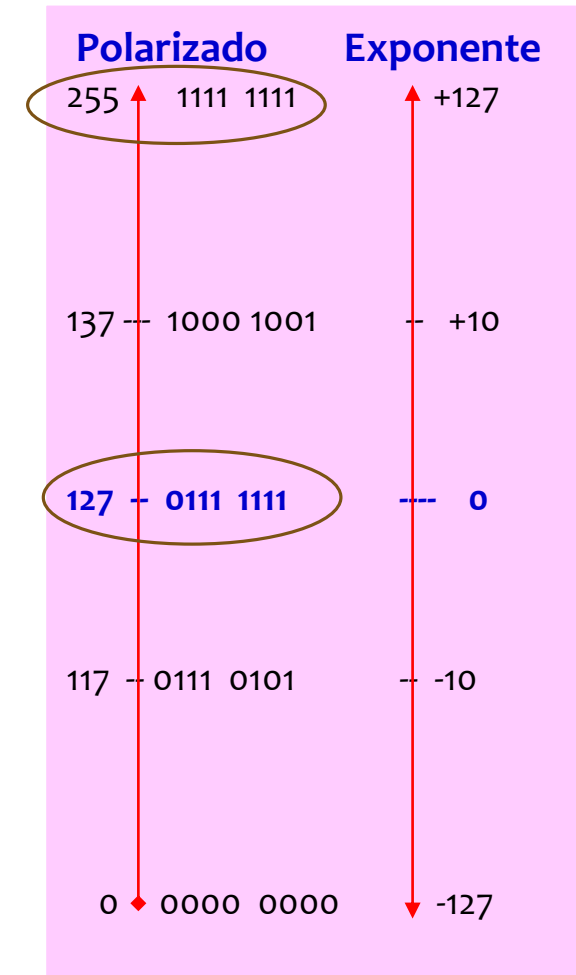
de exponente

Normalizado de mantisa

$$C = 127 \pm e ; 1 \leq m < 2)_d \rightarrow m = 1, b_1 b_2 b_3 \dots b_n$$

Precisión SIMPLE (32 blts) - EXCESO 127

Signo	Exponente	Mantisa			
0/1	Polarizado ("C")	Normalizada			
S	CCCC CCC	C	b ₁ b ₂ b ₃ b ₄ b ₅ b ₆ b ₇	b ₈ b ₉ b ₁₀ b ₁₁ b ₁₂ b ₁₃ b ₁₄ b ₁₅	b ₁₆ b ₂₃
1 byte		1 byte		1 byte	1 byte



Punto Flotante - Exceso 127

Los valores extremos que se utilizan para representar :

- *Nos permite manejar números desde ± 38 dígitos y lograr una exactitud entre 6 y 7 dígitos:*
- con $C = 1111\ 1111$ y $m=0$, se representa el $\pm$ infinito según el bits de signo
- con $C = 1111\ 1111$ y $m \neq 0$, se indica operaciones no validas, ej. 0.infinito
- con $C = 0000\ 0000$, $m=0$ y el bits de signo de valor 1 se representa el cero,
- con $C = 0000\ 0000$ y $m \neq 0$, el N_e esta desnormalizado.
- En definitiva el intervalo del exponente va de $- 128$ a $+ 127$

BANDERAS (FLAGS)

SIGNO: (S)

S=0 Resultado positivo.

S=1 Resultado negativo.

CERO: (Z)

Z=0 Resultado $\neq 0$

Z=1 Resultado = 0

CARRY: (C)

C=0 No existe acarreo.

C=1 Existe acarreo.

OVERFLOW: (V)

V=0 No existe overflow. (si sumando 2 nros. de igual signo y el resultado no cambia de signo, o si se suman 2 nros. de signos distintos)

V=1 Existe overflow. (si sumando 2 nros. de igual signo y resultado cambia de signo, quedando el resultado truncado)

BCD – Código de Representación Decimal

BCD es un acrónimo de *Código Binario Decimal*

- Los Códigos BCD son convenciones que permiten la representación de números decimales (0 a 9) en bloques binarios de 4 bits.
- Son códigos de representación de números denominados “Códigos Ponderados” porque adjudican cierto peso a los “1” binarios según la posición que ocupan en el bloque. (La suma de los pesos de cada combinación debe ser igual al número decimal representado).
- En esta notación se utilizan grupos de 4 bits para representar cada dígito decimal del 0 al 9. Con este método podemos representar dos dígitos por byte de información.
- Este método es mucho mas práctico para representación de números en la memoria en comparación al ASCII, todavía se queda por debajo del binario, ya que con un byte en el método BCD solo podemos representar dígitos del 0 al 99, en cambio, en formato binario podemos representar todos los dígitos desde 0 hasta 255.

BCD – Código de Representación Decimal

Casos de uso:

- Este formato es utilizado principalmente para representar números muy grandes en aplicaciones mercantiles ya que facilita las operaciones con los mismos evitando errores de redondeo.
- Sigue siendo ampliamente utilizado para almacenar datos, en aritmética binaria o en electrónica. Los números se pueden mostrar fácilmente en visualizadores de siete segmentos enviando cada cuarteto BCD a un visualizador.
- La BIOS de un ordenador personal almacena generalmente la fecha y la hora en formato BCD; probablemente por razones históricas se evitó la necesidad de su conversión en ASCII.

BCD – Código de Representación Decimal

La ventaja del código BCD frente a la representación binaria clásica es que no hay límite para el tamaño de un número. Los números que se representan en formato binario están generalmente limitados por el número mayor que se pueda representar con 8, 16, 32 o 64 bits.

Por el contrario, utilizando BCD, añadir un nuevo dígito solo implica añadir una nueva secuencia de 4 bits.

$$137_{10} = 10001001_2 \quad \text{Binario}$$

$$137_{10} = 0001 \ 0011 \ 0111 \quad \text{BCD}$$

BCD – Código de Representación Decimal

Existen varias clases de códigos BCD, veremos el código BCD Puro (8421)

Se denomina así porque los pesos en cada bloque coinciden con los 4 primeros pesos del sistema binario puro

VALOR DECIMAL	BCD PURO 8421
0	0000
1	0001
2	0010
3	0011
4	0100
5	0101
6	0110
7	0111
8	1000
9	1001

Sistemas de Representación

Unidades de medida para almacenamiento de datos – Bytes

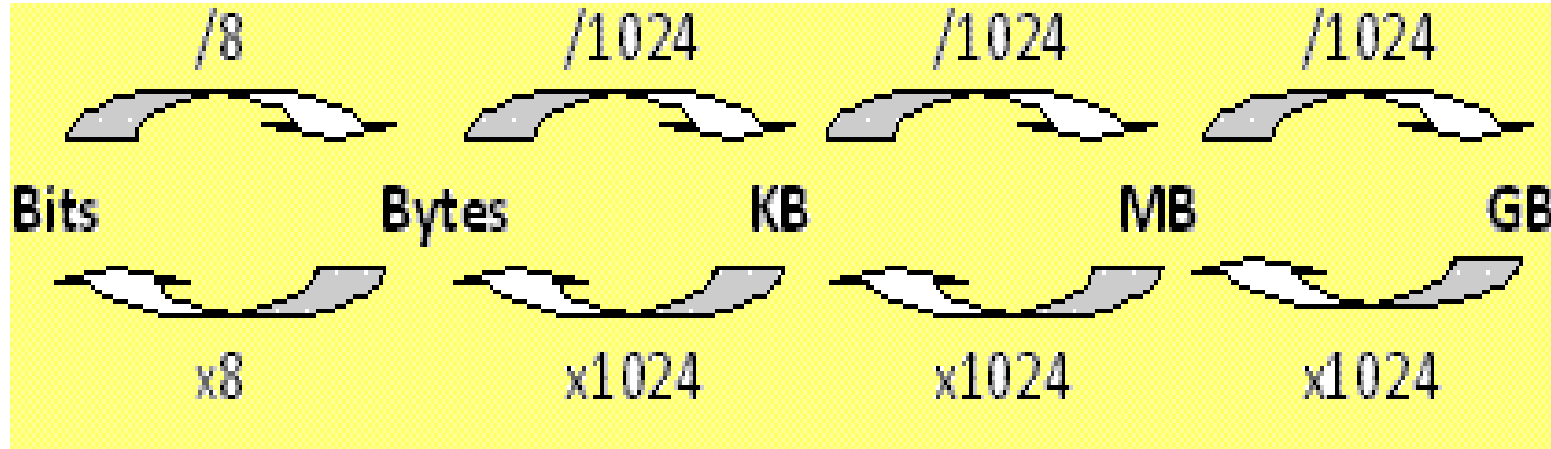
El sistema de medición para almacenamiento de datos se fundamenta en:

- Un Byte como medida base.
- 1024 (2^{10}) como factor multiplicador para el incremento.
- Los prefijos: Kilo, Mega, Giga, Tera, etc.

Así:

- 1 KiloByte = 1024 Bytes = 2^{10}
- 1 MegaByte = 1024 KiloBytes = 2^{20}
- 1 GigaByte = 1024 MegaBytes = 2^{30}
- 1 TeraByte = 1024 GigaBytes = 2^{40}

Conversión entre las Unidades de Información



Ejemplo:

Una película ocupa 1550 MB. ¿Cuántas películas caben en un DVD que tiene 4,7 GB de capacidad?

Solución:

Para pasar de MB a GB se divide una vez por 1024. **$1550 \text{ MB} / 1024 = 1,513671875 \text{ GB}$**
(Obtenemos lo que ocupa una película pero en GB)

Ahora dividimos la capacidad del DVD por lo que ocupa la película.

$4,7 \text{ GB} / 1,513671875 \text{ GB} = 3,1$ (Por lo tanto caben **3 películas en el DVD**)

Bibliografía Sugerida

- **Arquitectura de Computadoras. Patricia Quiroga. 1ra Edición**
 - ✓ Capítulos 2, 3 y 4
- **Organización y Arquitectura de Computadores. Stallings, William. 7ma Edición**
 - ✓ Capítulo 9
- **Introducción a la Organización y Arquitectura de la Computadora. Osio, Morales. 1ra Edición. UNAJ.**
 - ✓ Capítulo 1
- **Link de interés**
 - ✓ <http://coderstoolbox.net/number/>
 - ✓ <http://babbage.cs.qc.edu/IEEE-754/>